AI-BASED LEARNER STREAM RECOMMENDATION SYSTEM

Helping Grade 9 learners explore possible Grade 10 streams using marks, interests and career goals.

Student number	Initials	Surname
224917110	BO	Sebati
240192338	JH	Mdaka
223822094	NBS	Duma
240303598	R	Ramodise
240996666	MC	Ratsaka
224384481	TD	Mathiba

THE PROBLEM

THE CHALLENGE

- Grade 9 learners must choose Grade 10 academic subjects and consider possible future career paths.
- Some learners struggle to identify the stream that best matches their school performance, interests and career goals.
- Learners may have limited access to personalised guidance.
- Some do not understand how current subjects connect to future study and employment opportunities.

WHY IT MATTERS

- It can be difficult to compare available academic options.
- Learners may find it challenging to make informed decisions about their futures.
- Additional accessible guidance can help learners explore their options.

OUR SOLUTION

A WEB-BASED RECOMMENDATION SYSTEM

Combines machine learning, keyword-based text processing and a user-friendly interface to suggest a suitable academic stream.

01

ENTER MARKS

Learners enter their Grade 9 subject marks.

02

SHARE INTERESTS

Learners provide interests and career goals.

03

ANALYSE

Machine learning analyses marks; NLP identifies relevant keywords.

04

RECOMMEND

The system combines the available results.

05

VIEW RESULTS

The website displays a suggested stream and explanation.

The system supports educational guidance; it does not replace teachers or professional career counselling.

PROJECT OBJECTIVES

OBJECTIVES

- Prepare and inspect a dataset containing learner subject marks and recommended stream labels.
- Develop a Decision Tree model to predict an academic stream from subject marks.
- Develop a Random Forest model and compare its performance with the Decision Tree.
- Evaluate models using accuracy, precision, recall, F1-score and other appropriate methods.
- Develop a keyword-based NLP component to analyse interests and career goals.

OBJECTIVES (CONTINUED)

- Build a Flask website that collects learner information and displays recommendations.
- Integrate the machine learning model, NLP component and website.
- Test functionality, usability and reliability; identify limitations and improvements.
- Use GitHub and GitHub Projects to manage files and monitor progress.
- Prepare the final documentation, presentation and project poster.

SYSTEM REQUIREMENTS

FUNCTIONAL REQUIREMENTS

- Display a home page explaining the application.
- Allow entry of Grade 9 subject marks.
- Collect learners' interests and career goals.
- Support school selection where available.
- Process marks using the saved machine learning model.
- Analyse written responses using NLP keywords.
- Combine scores and display a suggested stream with an explanation.
- Show related career information and handle invalid input or errors.

NON-FUNCTIONAL REQUIREMENTS

- Usability: easy to understand and navigate.
- Performance: process submissions within a reasonable time.
- Reliability: handle valid input consistently and provide error feedback.
- Security: protect sensitive information and minimise personal data.
- Maintainability: organise code so components can be updated.
- Compatibility: test desktop and mobile screen sizes.
- Explainability: provide understandable recommendations, not guaranteed success probabilities.

BUSINESS SUCCESS CRITERIA

- Learners can access the website and navigate its pages.
- Valid subject marks can be submitted successfully.
- The application loads the saved model and generates a prediction.
- The chatbot processes written interests and career goals.
- The recommendation module combines the available outputs.
- The results page displays the recommendation and explanation.
- Functional tests are completed and documented.
- Model evaluation uses an appropriate testing approach.
- Data leakage concerns are investigated and reported.
- The GitHub repository contains the correct project files.
- Final documentation, presentation, poster and supporting evidence ar

PROJECT RISK MANAGEMENT

RISK	IMPACT	MITIGATION
Insufficient or unrepresentative data	High	Document data source and limitations; seek verified data where possible.
Low machine learning accuracy	High	Review data quality, labels, features and model configurations.
Data leakage	High	Check how labels and chatbot responses were created; evaluate with independent data.
Incorrect NLP keyword matches	Medium	Test wording, spelling variations and ambiguous responses.
Website and model integration errors	High	Test individual components and the complete learner journey.
Incorrect school information	Medium	Verify school information and indicate when details are unavailable.
Project delays	High	Monitor tasks and deadlines using GitHub Projects.
Security or privacy problems	High	Collect only necessary information and protect application secrets.
Poor usability	Medium	Test the interface and improve navigation and explanations.
Incomplete submission evidence	High	Check documentation, declaration, poster, presentation, Grammarly evidence and final PDF.

TOOLS AND TECHNIQUES

TOOLS AND TECHNOLOGIES

- Python: Main language for recommendation logic, data processing and Flask backend.
- Scikit-learn: Decision Tree, Random Forest, model evaluation and cross-validation.
- Pandas: Reads, inspects and prepares the CSV dataset.
- Joblib: Saves and loads the trained model and label encoder.
- Flask: Web application backend; handles routes and connects interface to recommendations.
- HTML / CSS / JavaScript: Page structure, visual design, layout and client-side interactions.
- Git and GitHub: Version control, collaboration and project evidence.

TECHNIQUES USED

- Decision Tree – predicts streams from subject marks.
- Random Forest – combines multiple decision trees.
- Natural Language Processing – keyword matching for interests and career goals.
- Data preparation – checks missing values, duplicates, mark ranges and labels.
- Model evaluation – accuracy, precision, recall and F1-score.
- Cross-validation – examines performance across training subsets.
- Recommendation integration – 60% machine learning and 40% NLP when keywords are found.
- Functional testing – checks navigation, validation, model loading and recommendations.

PROJECT POSTER

BUSINESS ANALYSIS 3.2 | AIBUY3A

AI-BASED LEARNER STREAM RECOMMENDATION SYSTEM

Helping Grade 9 learners explore possible Grade 10 streams

THE PROBLEM

Choosing a Grade 10 stream can be confusing when learners are unsure how their marks, interests and career goals relate to available options.

This project offers a simple starting point for exploring those options.

HOW IT WORKS

1. Select a school (optional)
2. Enter nine subject marks
3. Answer interest and career questions
4. Analyse marks and text
5. View a stream recommendation and explanation

CURRENT TEST RESULTS

Decision Tree: 41.17% marks-only accuracy

Random Forest: 41.38% marks-only accuracy

NLP-only: 100% on the current dataset validation

Combined: 99.9% on the current dataset validation

The high NLP and combined results may be inflated by templated responses and possible data leakage.

OUR AIM

Build a web application that uses nine Grade 9 subject marks and learner interests to suggest a possible stream.

The system supports learner choice. It does not replace teachers, parents or school counsellors.

TECHNOLOGY

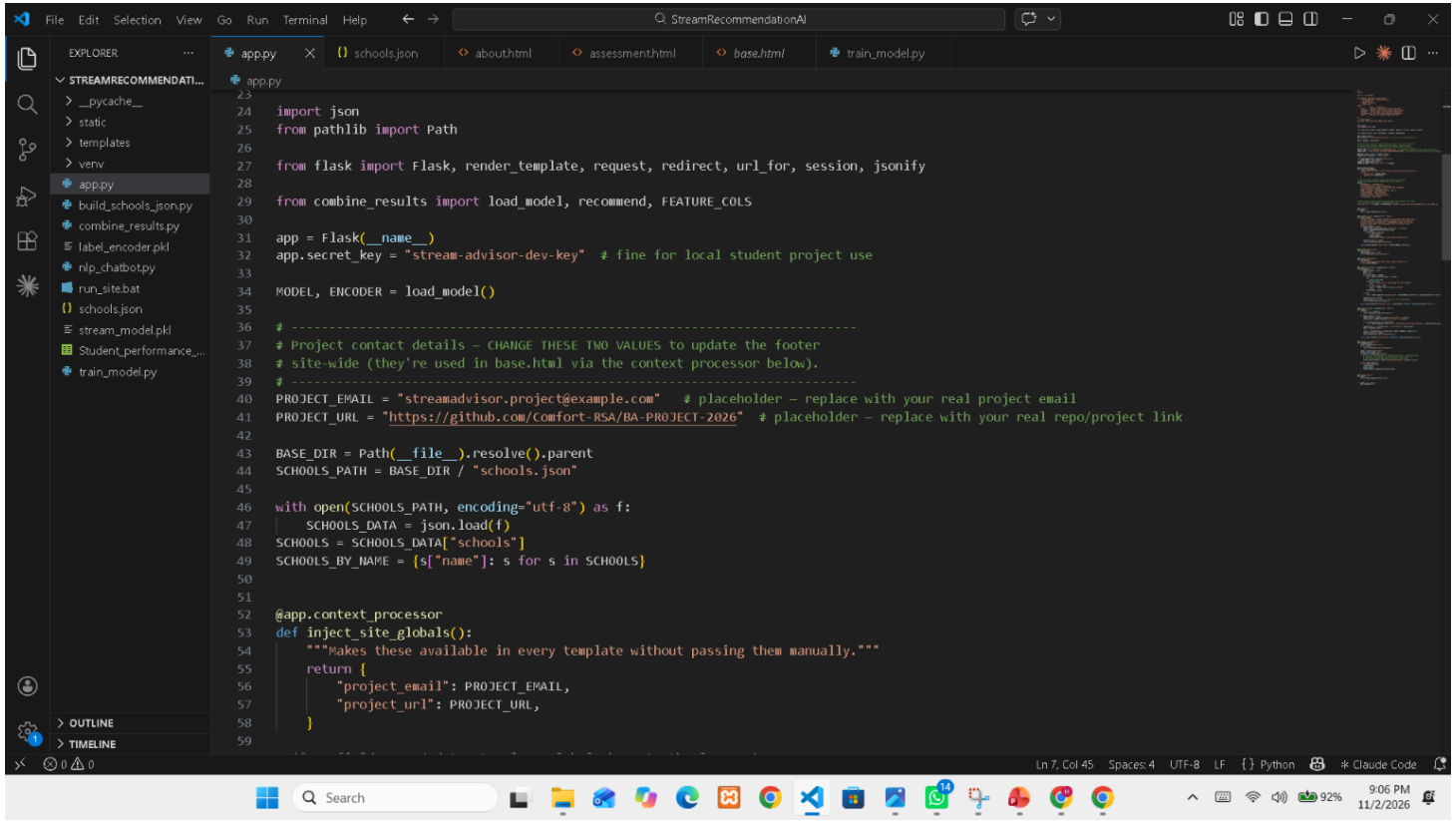
Python • Flask • scikit-learn • Pandas • HTML/CSS • JavaScript

Decision Tree and Random Forest classify marks. A keyword-based NLP component analyses interests and career goals. The current code combines model and NLP scores when relevant keywords are found.

NEXT STEPS

Check how the data and chatbot answers were created. Test with independent learner responses. Review errors by stream and improve the model based on evidence.

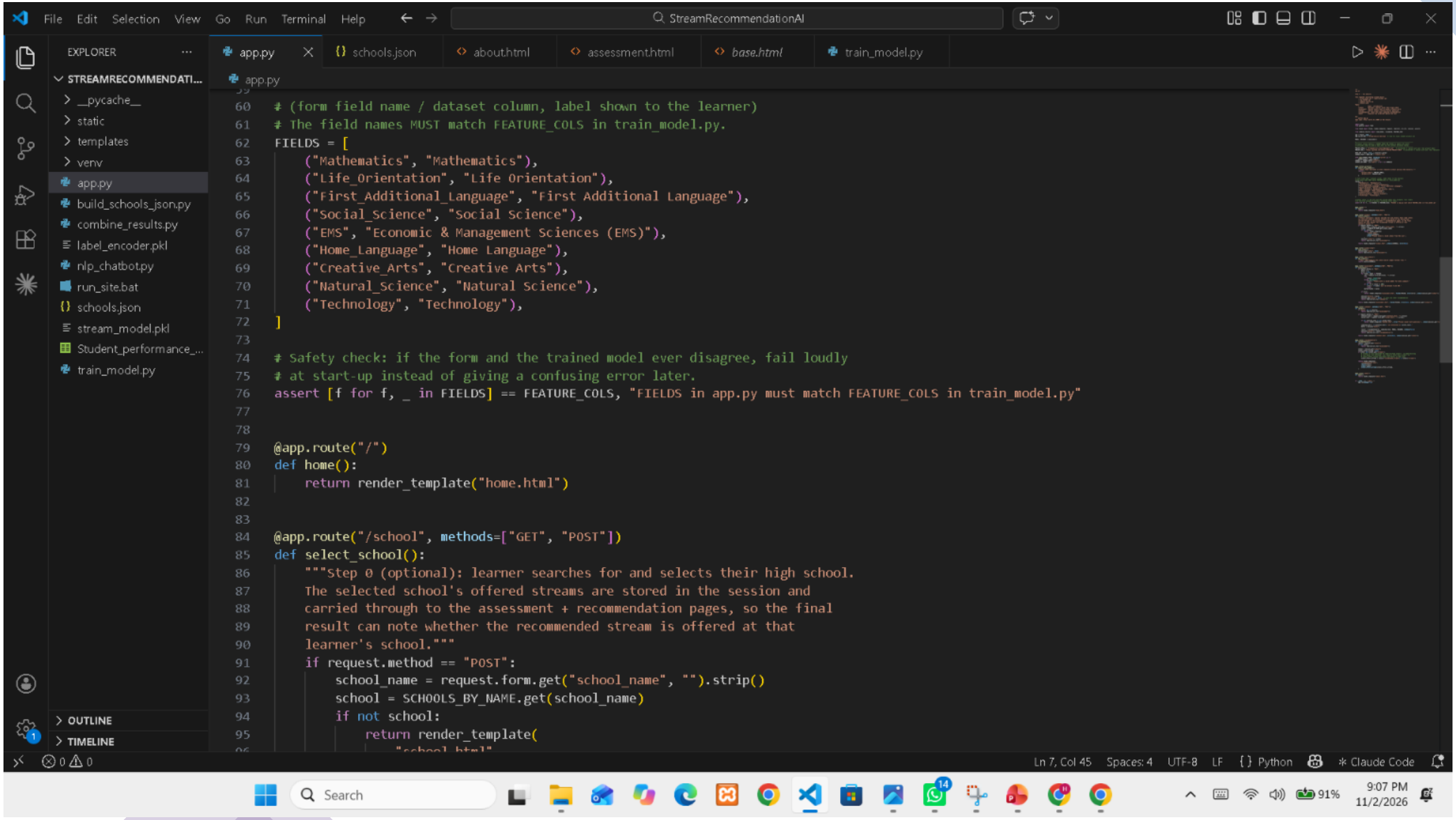
Time-series analysis and deep learning are not evidenced in the current code and must not be claimed as completed.

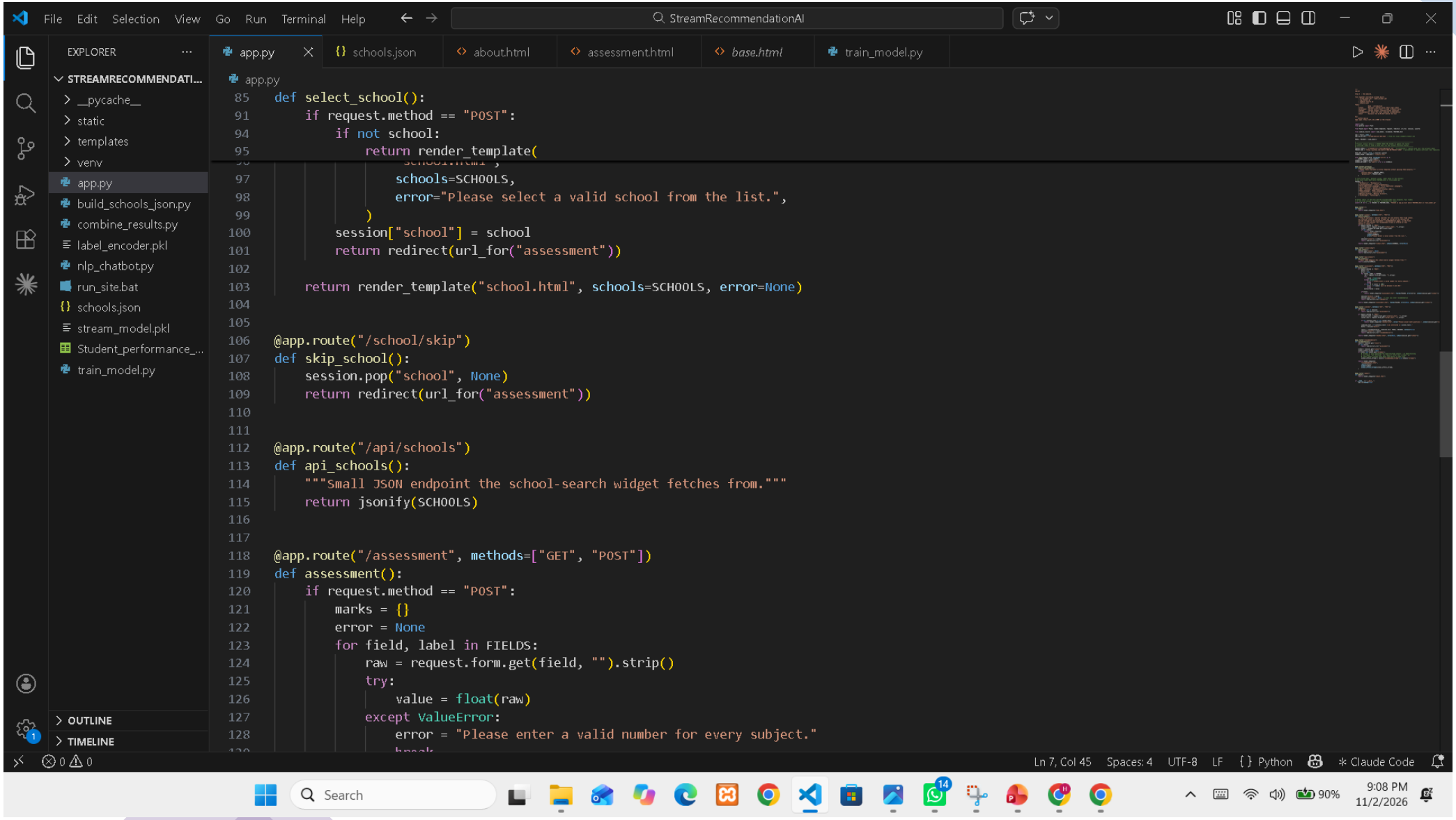

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
schools.json
about.html
assessment.html
base.html
train_model.py
STREAMRECOMMENDATI...
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
import json
from pathlib import Path
from flask import Flask, render_template, request, redirect, url_for, session, jsonify
from combine_results import load_model, recommend, FEATURE_COLS
app = Flask(__name__)
app.secret_key = "stream-advisor-dev-key" # fine for local student project use
MODEL, ENCODER = load_model()

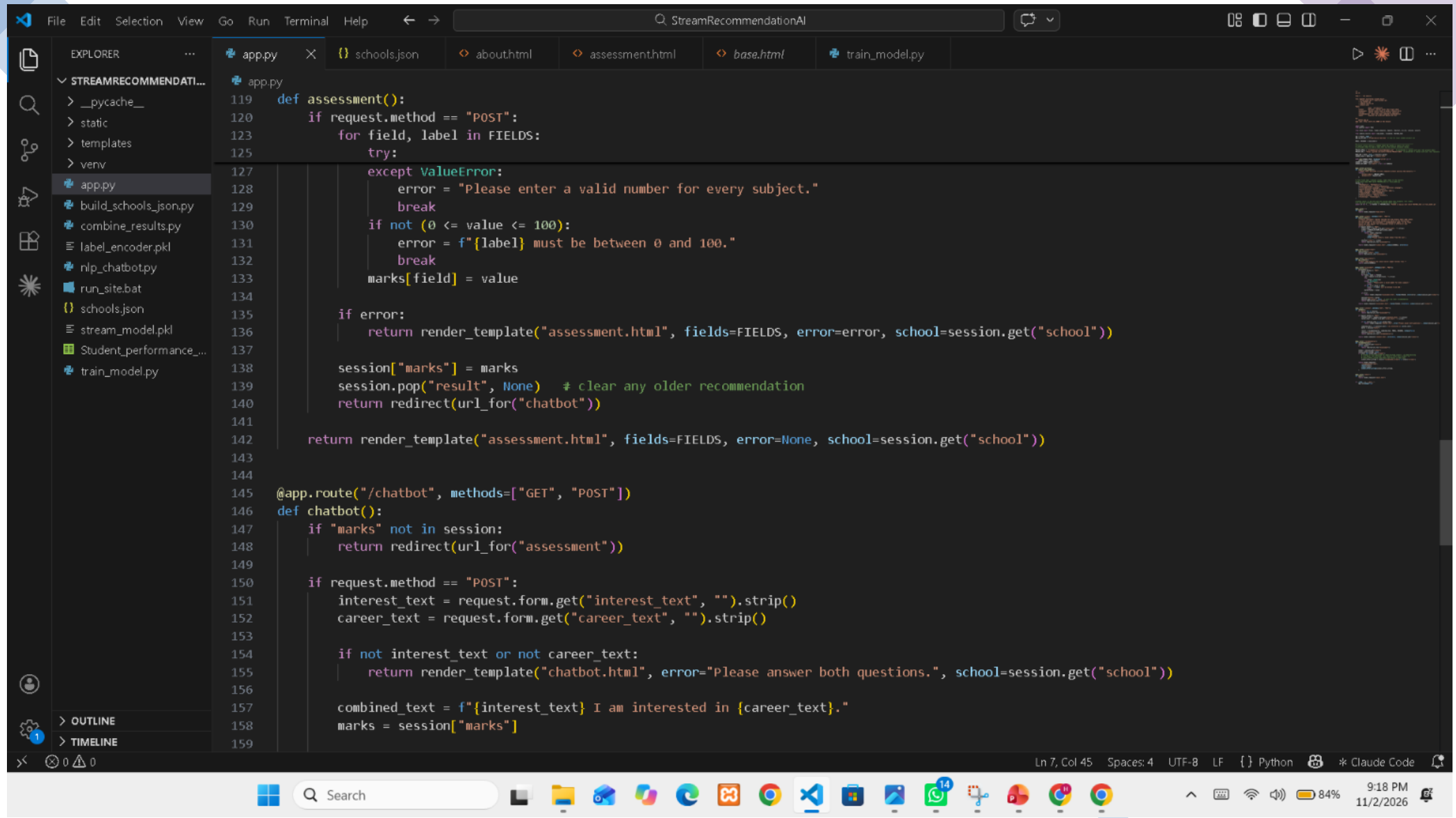
Project contact details — CHANGE THESE TWO VALUES to update the footer
site-wide (they're used in base.html via the context processor below).

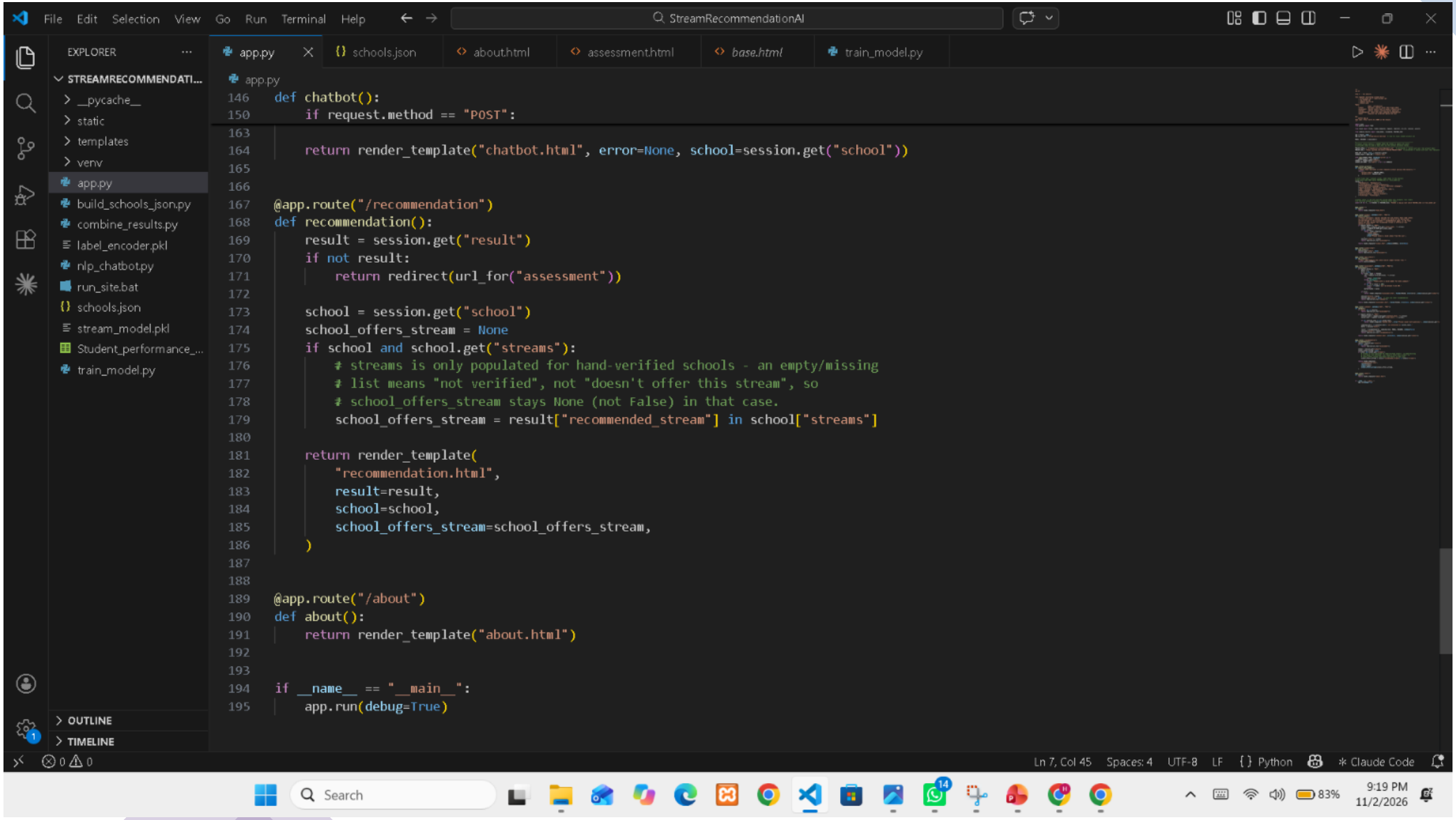
PROJECT_EMAIL = "streamadvisor.project@example.com"   # placeholder — replace with your real project email
PROJECT_URL = "https://github.com/Comfort-RSA/BA-PROJECT-2026" # placeholder — replace with your real repo/project link
BASE_DIR = Path(__file__).resolve().parent
SCHOOLS_PATH = BASE_DIR / "schools.json"
with open(SCHOOLS_PATH, encoding="utf-8") as f:
SCHOOLS_DATA = json.load(f)
SCHOOLS = SCHOOLS_DATA["schools"]
SCHOOLS_BY_NAME = {s["name"]: s for s in SCHOOLS}
@app.context_processor
def inject_site_globals():
"""Makes these available in every template without passing them manually."""
return {
"project_email": PROJECT_EMAIL,
"project_url": PROJECT_URL,
}
Ln 7, Col 45 Spaces: 4 UTF-8 LF Python Claude Code
Search
92%
9:06 PM
11/2/2026

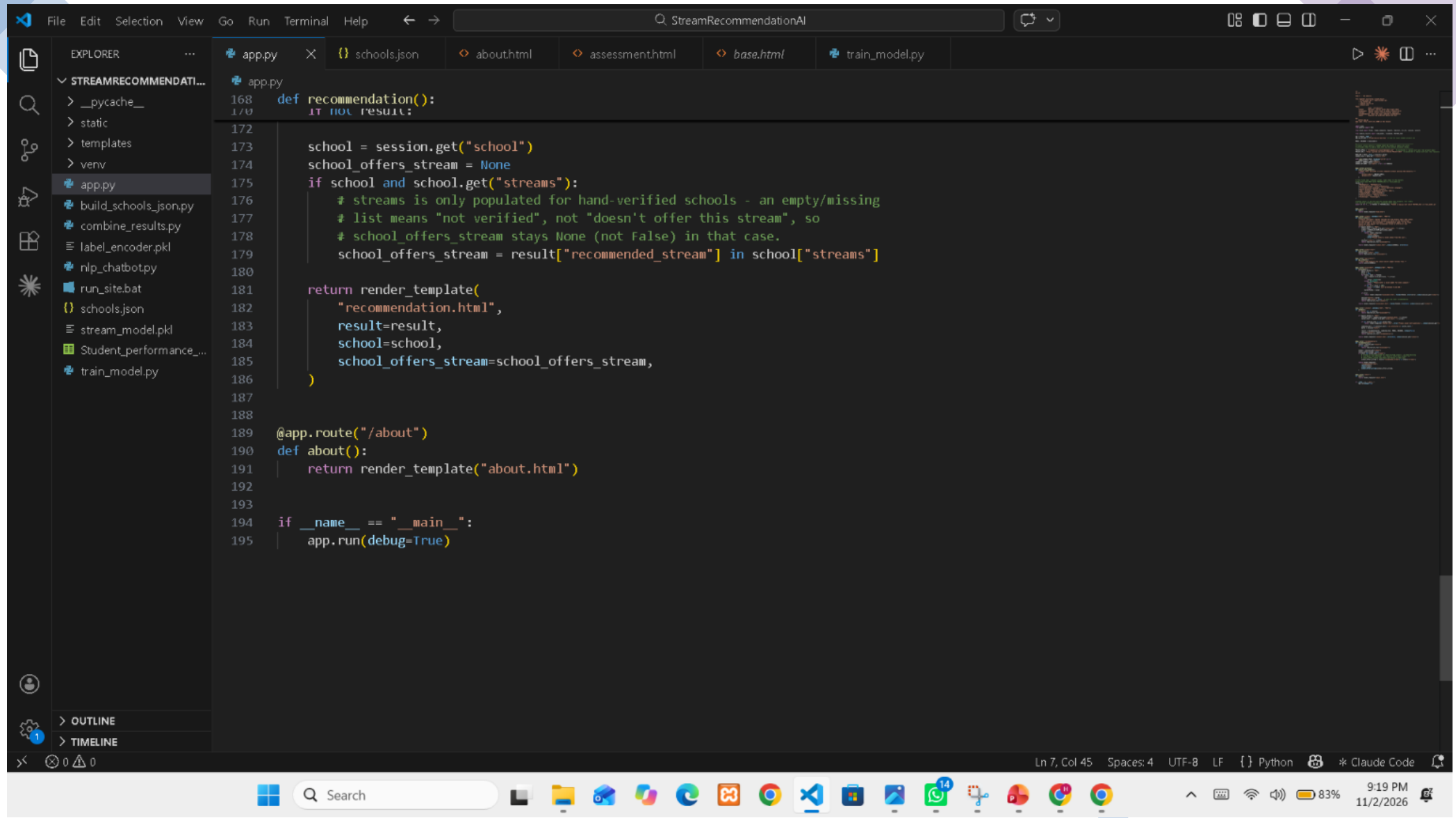
App.py

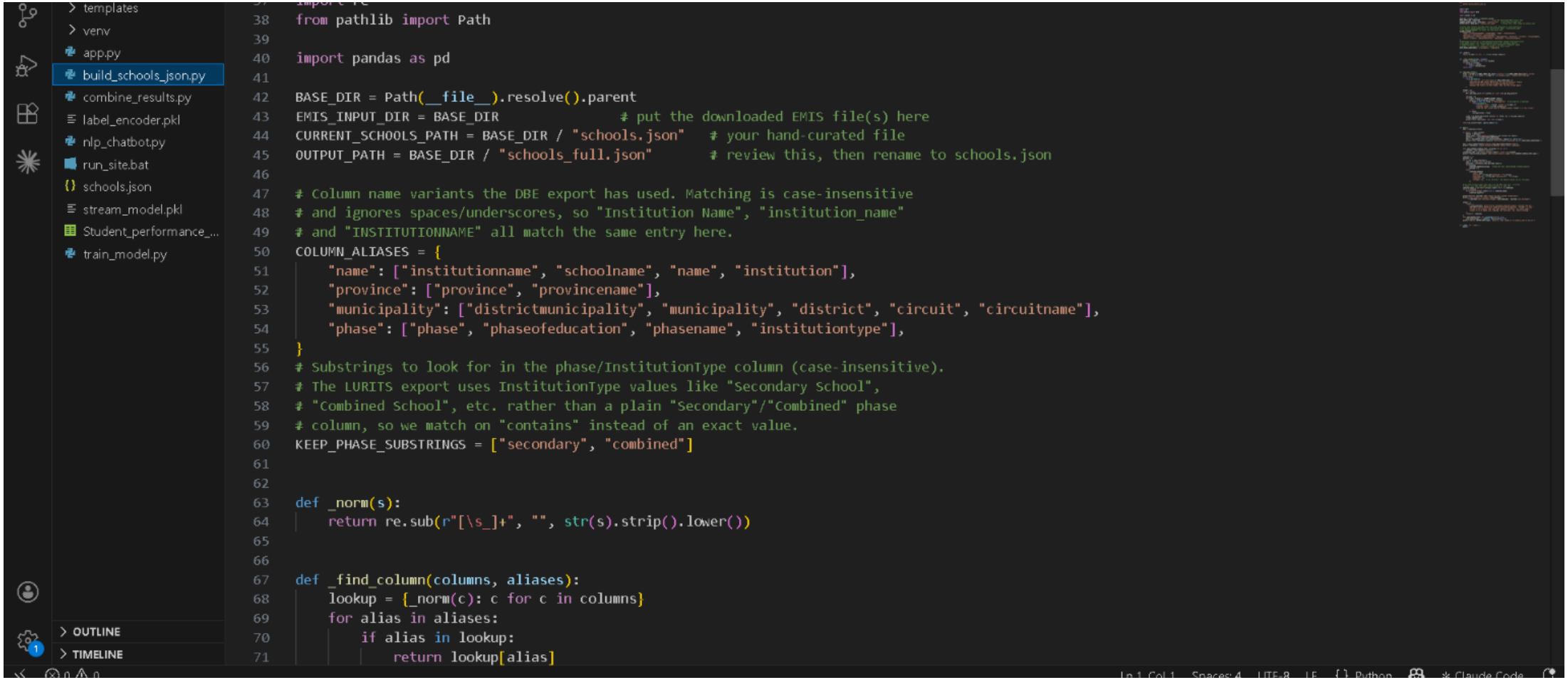

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
schools.json
about.html
assessment.html
base.html
train_model.py
STREAMRECOMMENDATI...
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
(form field name / dataset column, label shown to the learner)
The field names MUST match FEATURE_COLS in train_model.py.
FIELDS = [
("Mathematics", "Mathematics"),
("Life_Orientation", "Life Orientation"),
("First_Additional_Language", "First Additional Language"),
("Social_Science", "Social Science"),
("EMS", "Economic & Management Sciences (EMS)"),
("Home_Language", "Home Language"),
("Creative_Arts", "Creative Arts"),
("Natural_Science", "Natural Science"),
("Technology", "Technology"),
]
Safety check: if the form and the trained model ever disagree, fail loudly
at start-up instead of giving a confusing error later.
assert [f for f, _ in FIELDS] == FEATURE_COLS, "FIELDS in app.py must match FEATURE_COLS in train_model.py"
@app.route("/")
def home():
return render_template("home.html")
@app.route("/school", methods=["GET", "POST"])
def select_school():
"""Step 0 (optional): learner searches for and selects their high school.
The selected school's offered streams are stored in the session and
carried through to the assessment + recommendation pages, so the final
result can note whether the recommended stream is offered at that
learner's school."""
if request.method == "POST":
school_name = request.form.get("school_name", "").strip()
school = SCHOOLS_BY_NAME.get(school_name)
if not school:
return render_template(
Ln 7, Col 45 Spaces: 4 UTF-8 LF Python Claude Code
Search
91%
9:07 PM
11/2/2026


File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
schools.json
about.html
assessment.html
base.html
train_model.py
STREAMRECOMMENDATI...
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def select_school():
if request.method == "POST":
if not school:
return render_template(
schools=SCHOOLS,
error="Please select a valid school from the list.",
)
session["school"] = school
return redirect(url_for("assessment"))
return render_template("school.html", schools=SCHOOLS, error=None)
@app.route("/school/skip")
def skip_school():
session.pop("school", None)
return redirect(url_for("assessment"))
@app.route("/api/schools")
def api_schools():
"""Small JSON endpoint the school-search widget fetches from."""
return jsonify(SCHOOLS)
@app.route("/assessment", methods=["GET", "POST"])
def assessment():
if request.method == "POST":
marks = {}
error = None
for field, label in FIELDS:
raw = request.form.get(field, "").strip()
try:
value = float(raw)
except ValueError:
error = "Please enter a valid number for every subject."
Ln 7, Col 45
Spaces: 4
UTF-8
LF
Python
Claude Code
Search
90%
9:08 PM
11/2/2026

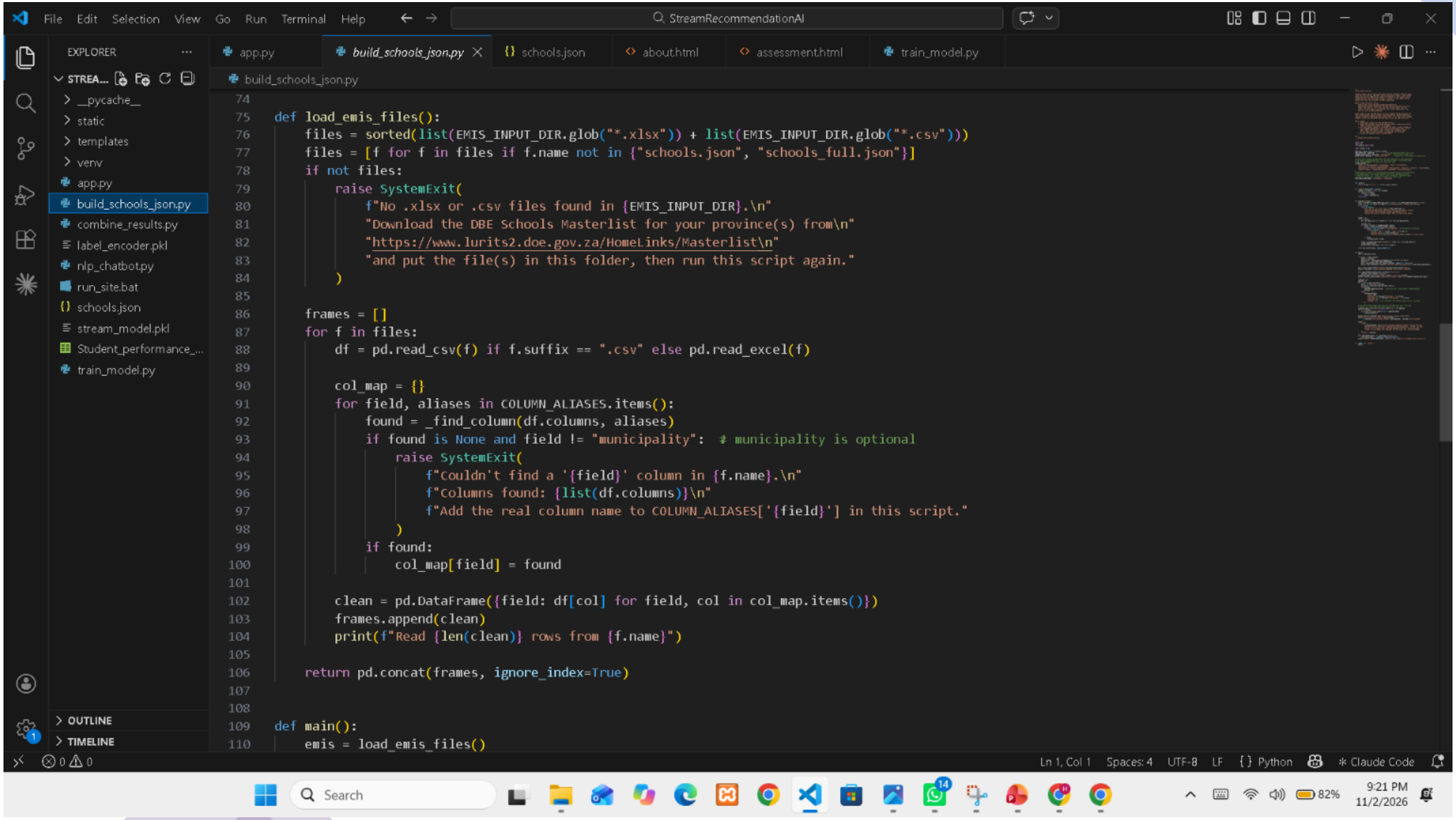

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
STREAMRECOMMENDATI...
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
app.py
schools.json
about.html
assessment.html
base.html
train_model.py
app.py
def assessment():
if request.method == "POST":
for field, label in FIELDS:
try:
except ValueError:
error = "Please enter a valid number for every subject."
break
if not (0 <= value <= 100):
error = f"{label} must be between 0 and 100."
break
marks[field] = value
if error:
return render_template("assessment.html", fields=FIELDS, error=error, school=session.get("school"))
session["marks"] = marks
session.pop("result", None) # clear any older recommendation
return redirect(url_for("chatbot"))
return render_template("assessment.html", fields=FIELDS, error=None, school=session.get("school"))
@app.route("/chatbot", methods=["GET", "POST"])
def chatbot():
if "marks" not in session:
return redirect(url_for("assessment"))
if request.method == "POST":
interest_text = request.form.get("interest_text", "").strip()
career_text = request.form.get("career_text", "").strip()
if not interest_text or not career_text:
return render_template("chatbot.html", error="Please answer both questions.", school=session.get("school"))
combined_text = f"{interest_text} I am interested in {career_text}."
marks = session["marks"]
Ln 7, Col 45 Spaces: 4 UTF-8 LF Python Claude Code
Search
84%
9:18 PM
11/2/2026

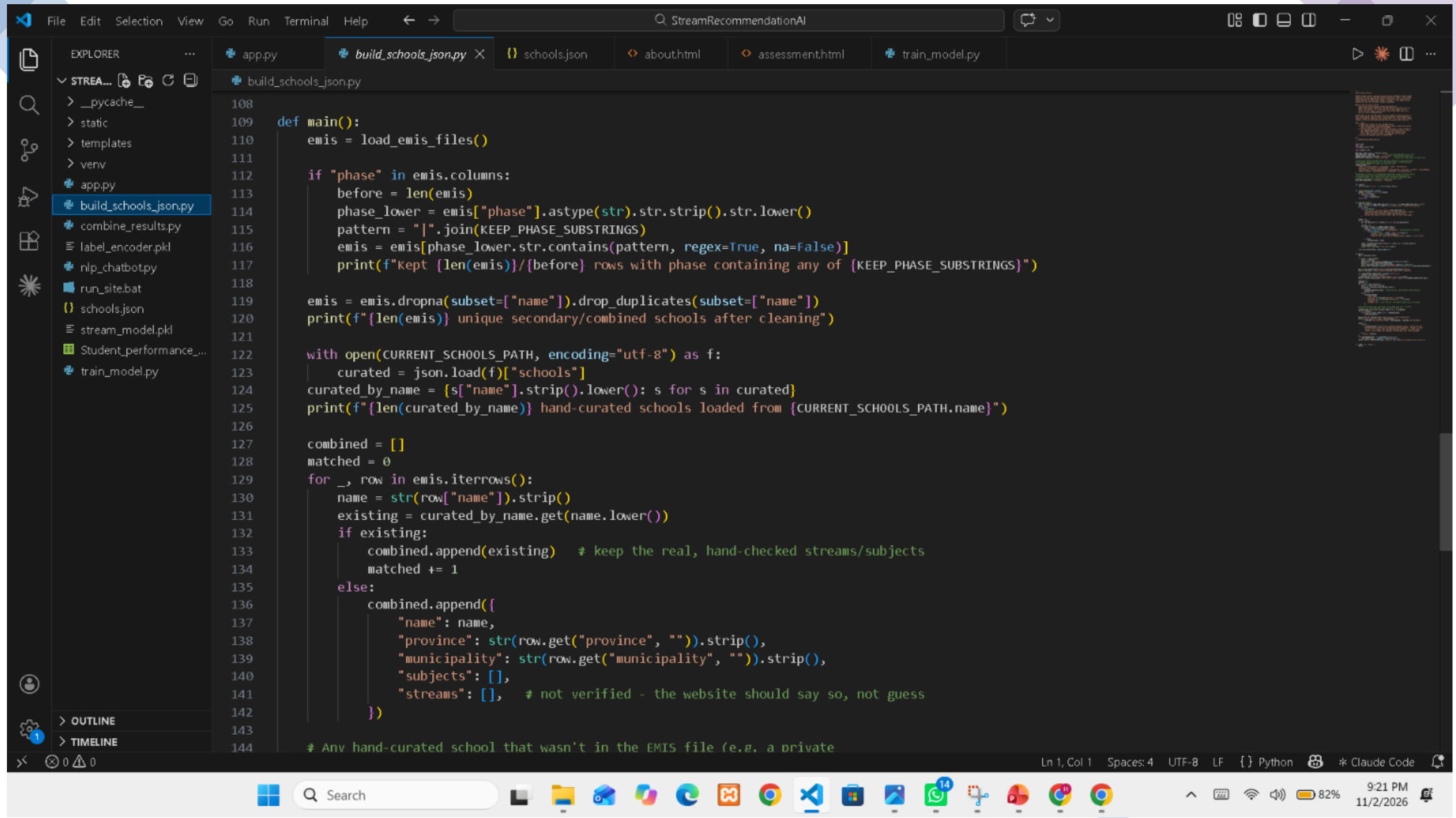

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
schools.json
about.html
assessment.html
base.html
train_model.py
STREAMRECOMMENDATI...
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
146 def chatbot():
150 if request.method == "POST":
163
164 return render_template("chatbot.html", error=None, school=session.get("school"))
165
166
167 @app.route("/recommendation")
168 def recommendation():
169 result = session.get("result")
170 if not result:
171 return redirect(url_for("assessment"))
172
173 school = session.get("school")
174 school_offers_stream = None
175 if school and school.get("streams"):
176 # streams is only populated for hand-verified schools - an empty/missing
177 # list means "not verified", not "doesn't offer this stream", so
178 # school_offers_stream stays None (not False) in that case.
179 school_offers_stream = result["recommended_stream"] in school["streams"]
180
181 return render_template(
182 "recommendation.html",
183 result=result,
184 school=school,
185 school_offers_stream=school_offers_stream,
186)
187
188
189 @app.route("/about")
190 def about():
191 return render_template("about.html")
192
193
194 if __name__ == "__main__":
195 app.run(debug=True)
Ln 7, Col 45 Spaces: 4 UTF-8 LF Python Claude Code
Search
83%
9:19 PM
11/2/2026

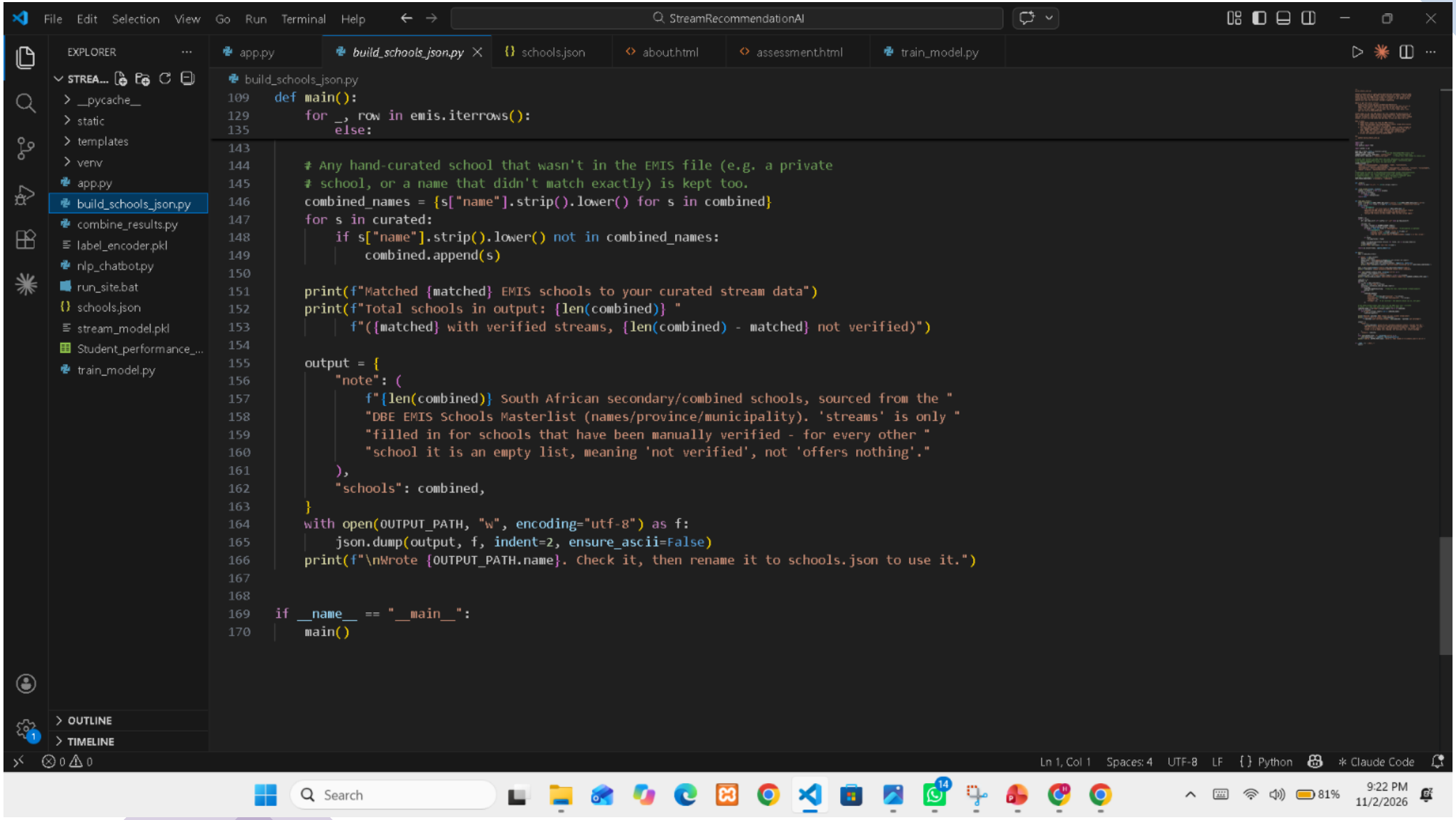

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
STREAMRECOMMENDATI...
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
app.py
schools.json
about.html
assessment.html
base.html
train_model.py
app.py
def recommendation():
if not result:
school = session.get("school")
school_offers_stream = None
if school and school.get("streams"):
streams is only populated for hand-verified schools - an empty/missing
list means "not verified", not "doesn't offer this stream", so
school_offers_stream stays None (not False) in that case.
school_offers_stream = result["recommended_stream"] in school["streams"]
return render_template(
"recommendation.html",
result=result,
school=school,
school_offers_stream=school_offers_stream,
)
@app.route("/about")
def about():
return render_template("about.html")
if __name__ == "__main__":
app.run(debug=True)
OUTLINE
TIMELINE
Ln 7, Col 45
Spaces: 4
UTF-8
LF
Python
Claude Code
Search
83%
9:19 PM
11/2/2026

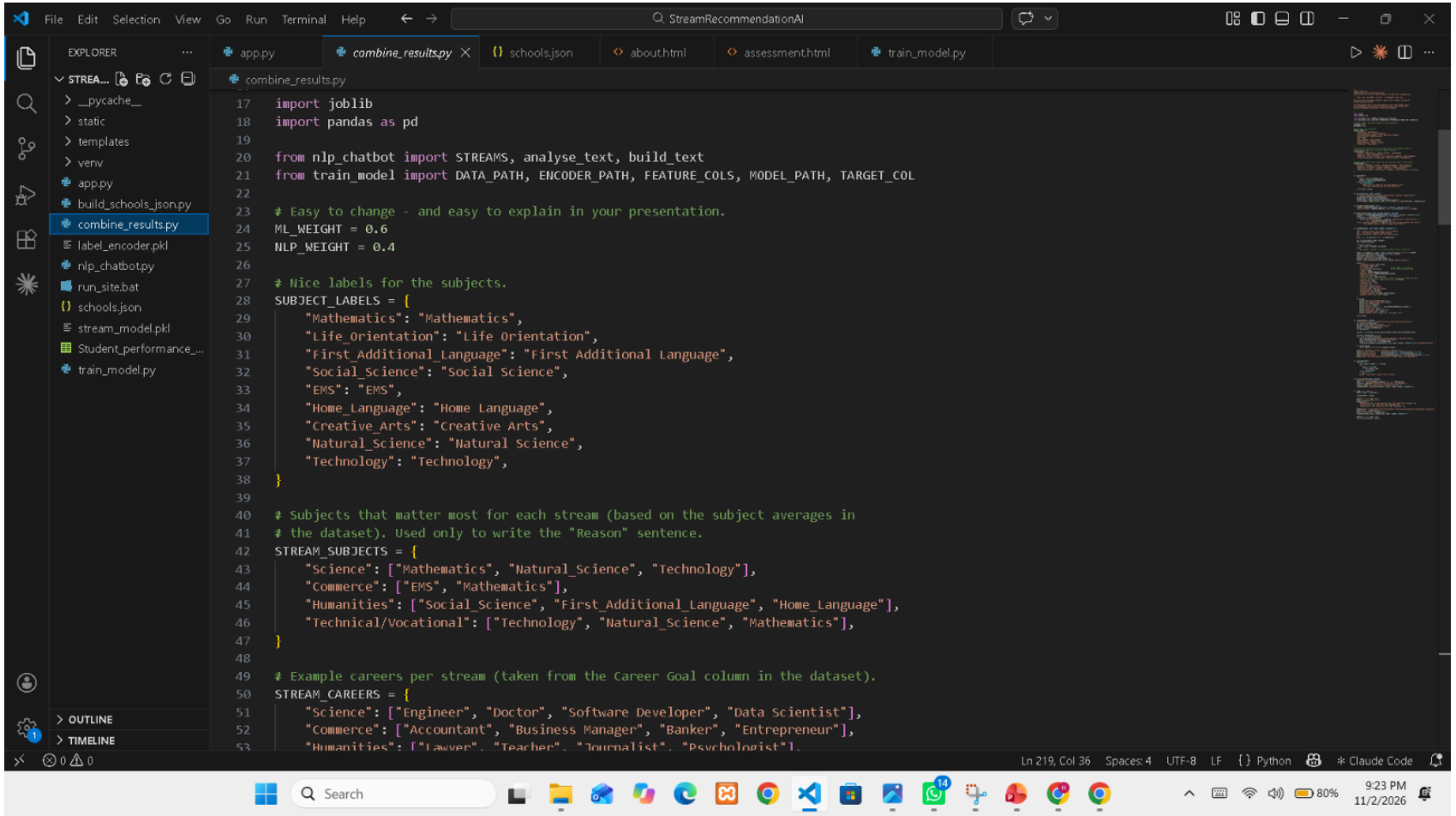


build_schools_json.py

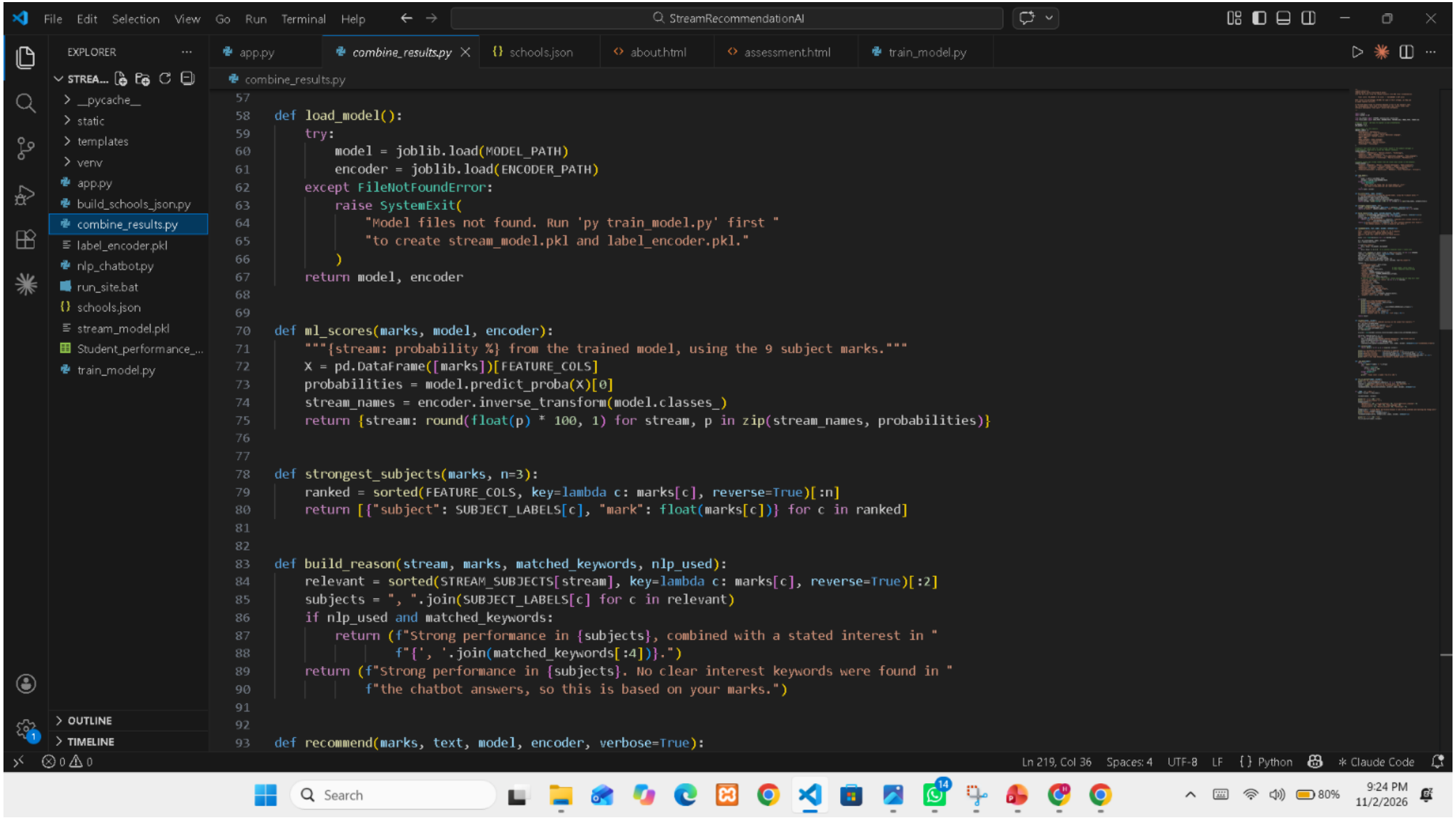

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
build_schools_json.py
schools.json
about.html
assessment.html
train_model.py
build_schools_json.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
74
75 def load_emis_files():
76 files = sorted(list(EMIS_INPUT_DIR.glob("*.xlsx")) + list(EMIS_INPUT_DIR.glob("*.csv")))
77 files = [f for f in files if f.name not in {"schools.json", "schools_full.json"}]
78 if not files:
79 raise SystemExit(
80 f"No .xlsx or .csv files found in {EMIS_INPUT_DIR}.\n"
81 "Download the DBE Schools Masterlist for your province(s) from\n"
82 "https://www.lurits2.doe.gov.za/HomeLinks/Masterlist\n"
83 "and put the file(s) in this folder, then run this script again."
84)
85
86 frames = []
87 for f in files:
88 df = pd.read_csv(f) if f.suffix == ".csv" else pd.read_excel(f)
89
90 col_map = {}
91 for field, aliases in COLUMN_ALIASES.items():
92 found = _find_column(df.columns, aliases)
93 if found is None and field != "municipality": # municipality is optional
94 raise SystemExit(
95 f"Couldn't find a '{field}' column in {f.name}.\n"
96 f"Columns found: {list(df.columns)}\n"
97 f"Add the real column name to COLUMN_ALIASES['{field}'] in this script."
98)
99 if found:
100 col_map[field] = found
101
102 clean = pd.DataFrame({field: df[col] for field, col in col_map.items()})
103 frames.append(clean)
104 print(f"Read {len(clean)} rows from {f.name}")
105
106 return pd.concat(frames, ignore_index=True)
107
108
109 def main():
110 emis = load_emis_files()
Ln 1, Col 1 Spaces: 4 UTF-8 LF Python Claude Code
Search
82%
9:21 PM
11/2/2026

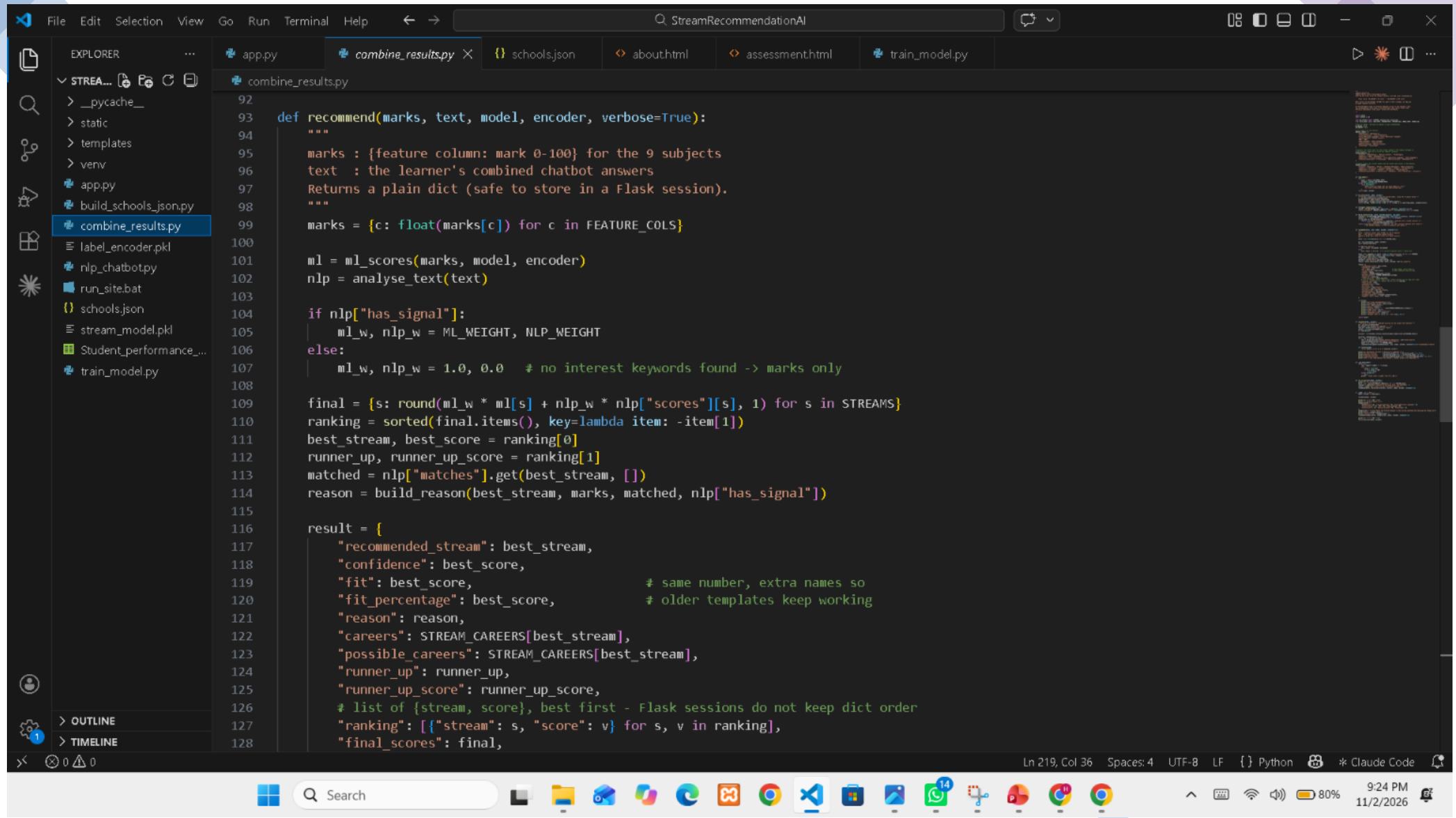

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
build_schools_json.py
schools.json
about.html
assessment.html
train_model.py
build_schools_json.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def main():
emis = load_emis_files()
if "phase" in emis.columns:
before = len(emis)
phase_lower = emis["phase"].astype(str).str.strip().str.lower()
pattern = "|".join(KEEP_PHASE_SUBSTRINGS)
emis = emis[phase_lower.str.contains(pattern, regex=True, na=False)]
print(f"Kept {len(emis)}/{before} rows with phase containing any of {KEEP_PHASE_SUBSTRINGS}")
emis = emis.dropna(subset=["name"]).drop_duplicates(subset=["name"])
print(f"{len(emis)} unique secondary/combined schools after cleaning")
with open(CURRENT_SCHOOLS_PATH, encoding="utf-8") as f:
curated = json.load(f)["schools"]
curated_by_name = {s["name"].strip().lower(): s for s in curated}
print(f"{len(curated_by_name)} hand-curated schools loaded from {CURRENT_SCHOOLS_PATH.name}")
combined = []
matched = 0
for _, row in emis.iterrows():
name = str(row["name"]).strip()
existing = curated_by_name.get(name.lower())
if existing:
combined.append(existing) # keep the real, hand-checked streams/subjects
matched += 1
else:
combined.append({
"name": name,
"province": str(row.get("province", "")).strip(),
"municipality": str(row.get("municipality", "")).strip(),
"subjects": [],
"streams": [], # not verified - the website should say so, not guess
})
Any hand-curated school that wasn't in the EMIS file (e.g. a private
Ln 1, Col 1 Spaces: 4 UTF-8 LF Python Claude Code
Search
82%
9:21 PM
11/2/2026

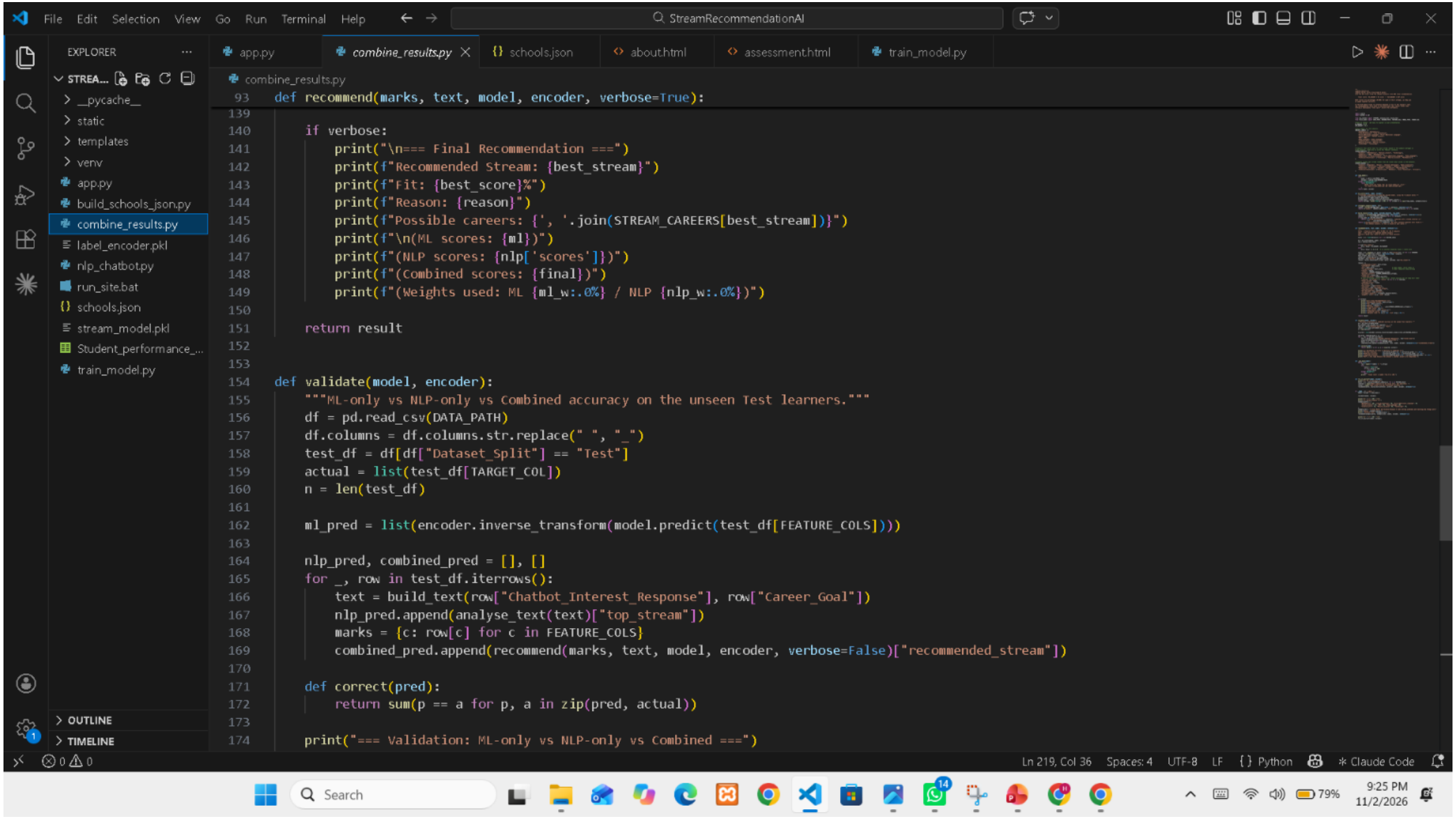

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
build_schools_json.py
schools.json
about.html
assessment.html
train_model.py
build_schools_json.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def main():
for _, row in emis.iterrows():
else:
Any hand-curated school that wasn't in the EMIS file (e.g. a private
school, or a name that didn't match exactly) is kept too.
combined_names = {s["name"].strip().lower() for s in combined}
for s in curated:
if s["name"].strip().lower() not in combined_names:
combined.append(s)
print(f"Matched {matched} EMIS schools to your curated stream data")
print(f"Total schools in output: {len(combined)} "
f"({matched} with verified streams, {len(combined) - matched} not verified)")
output = {
"note": (
f"{len(combined)} South African secondary/combined schools, sourced from the "
"DBE EMIS Schools Masterlist (names/province/municipality). 'streams' is only "
"filled in for schools that have been manually verified - for every other "
"school it is an empty list, meaning 'not verified', not 'offers nothing'."
),
"schools": combined,
}
with open(OUTPUT_PATH, "w", encoding="utf-8") as f:
json.dump(output, f, indent=2, ensure_ascii=False)
print(f"\nWrote {OUTPUT_PATH.name}. Check it, then rename it to schools.json to use it.")
if __name__ == "__main__":
main()
Ln 1, Col 1 Spaces: 4 UTF-8 LF Python Claude Code
Search
81%
9:22 PM
11/2/2026

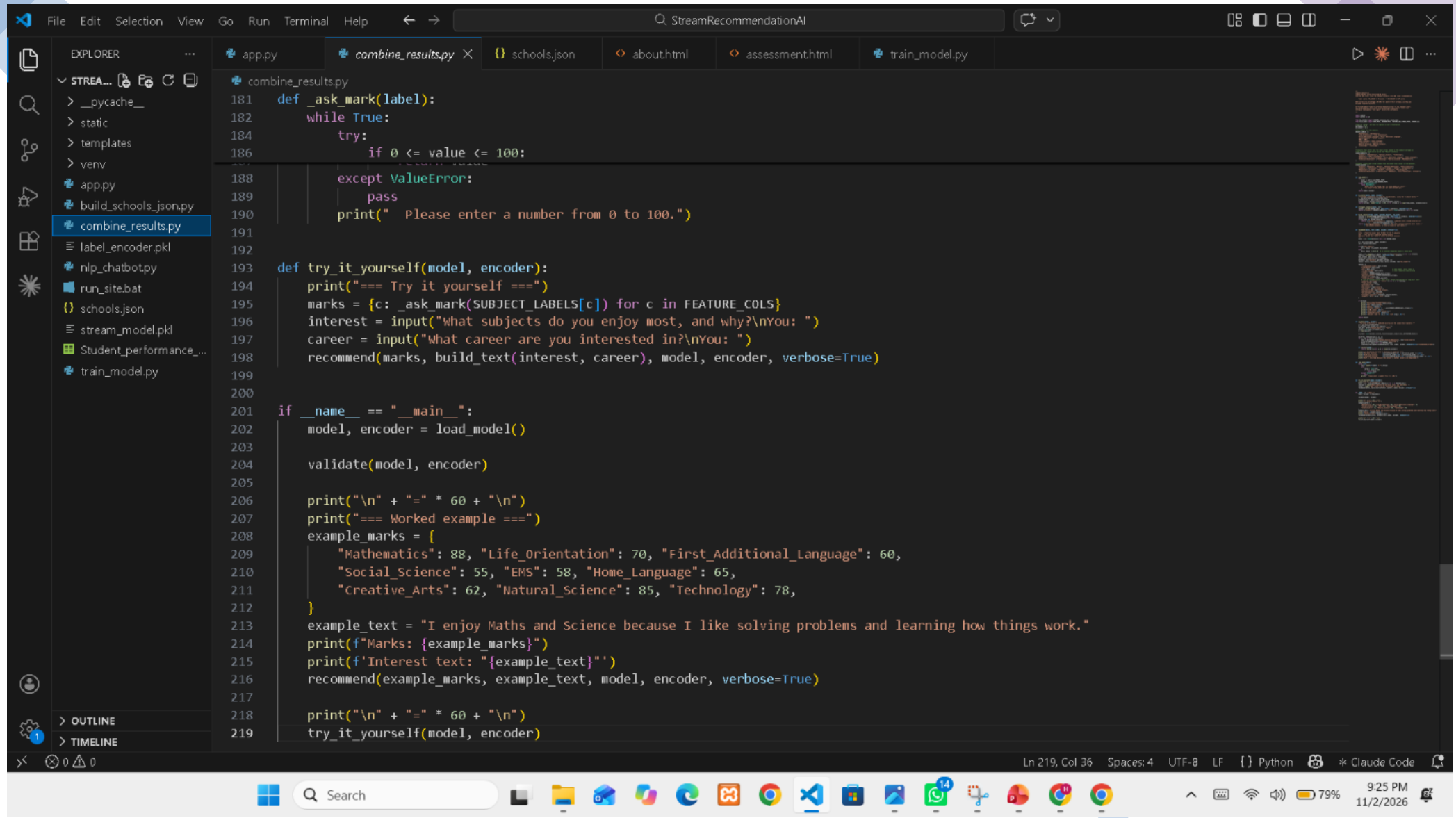


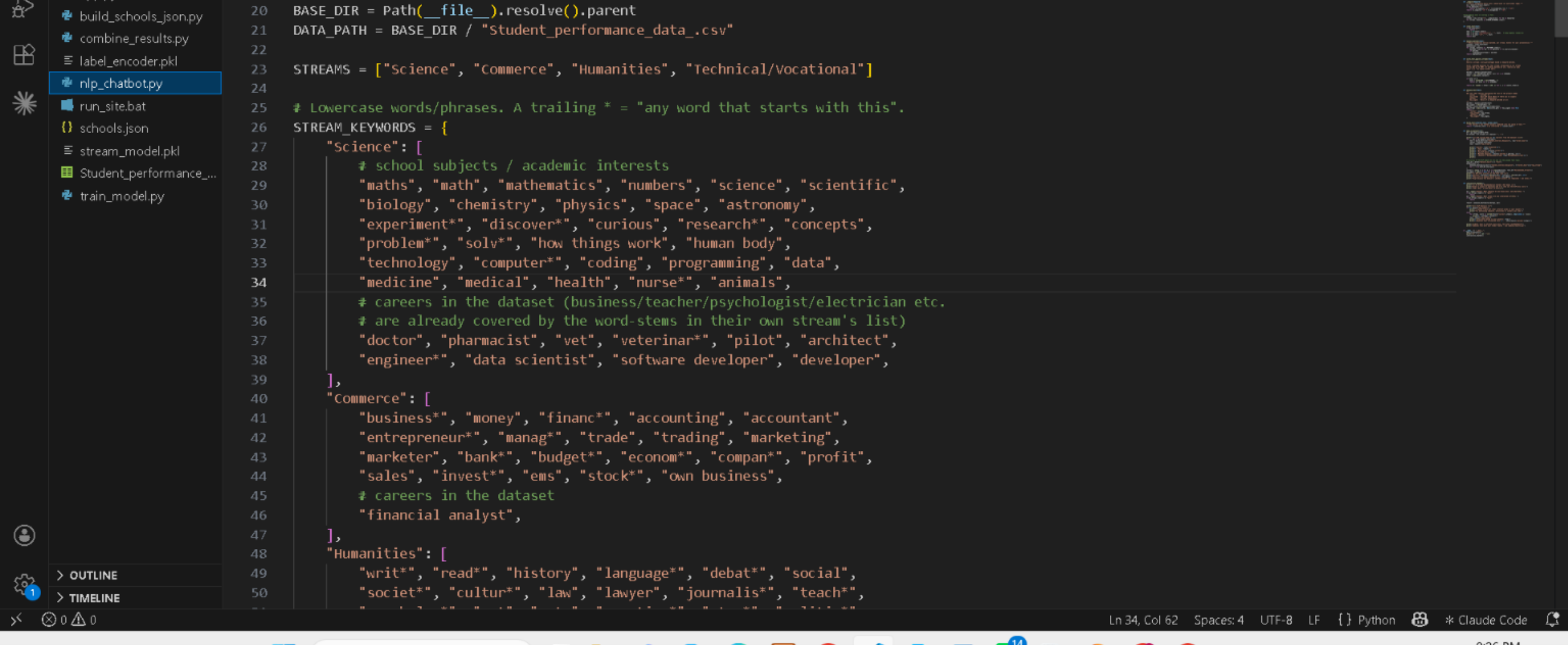
combine_results.py


File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
combine_results.py
schools.json
about.html
assessment.html
train_model.py
combine_results.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def load_model():
try:
model = joblib.load(MODEL_PATH)
encoder = joblib.load(ENCODER_PATH)
except FileNotFoundError:
raise SystemExit(
"Model files not found. Run 'py train_model.py' first "
"to create stream_model.pkl and label_encoder.pkl."
)
return model, encoder
def ml_scores(marks, model, encoder):
"""{stream: probability %} from the trained model, using the 9 subject marks."""
X = pd.DataFrame([marks])[FEATURE_COLS]
probabilities = model.predict_proba(X)[0]
stream_names = encoder.inverse_transform(model.classes_)
return {stream: round(float(p) * 100, 1) for stream, p in zip(stream_names, probabilities)}
def strongest_subjects(marks, n=3):
ranked = sorted(FEATURE_COLS, key=lambda c: marks[c], reverse=True)[:n]
return [{"subject": SUBJECT_LABELS[c], "mark": float(marks[c])} for c in ranked]
def build_reason(stream, marks, matched_keywords, nlp_used):
relevant = sorted(STREAM_SUBJECTS[stream], key=lambda c: marks[c], reverse=True)[:2]
subjects = ", ".join(SUBJECT_LABELS[c] for c in relevant)
if nlp_used and matched_keywords:
return (f"Strong performance in {subjects}, combined with a stated interest in "
f"{', '.join(matched_keywords[:4])}.")
return (f"Strong performance in {subjects}. No clear interest keywords were found in "
f"the chatbot answers, so this is based on your marks.")
def recommend(marks, text, model, encoder, verbose=True):
Ln 219, Col 36 Spaces: 4 UTF-8 LF Python Claude Code
Search
80%
9:24 PM
11/2/2026

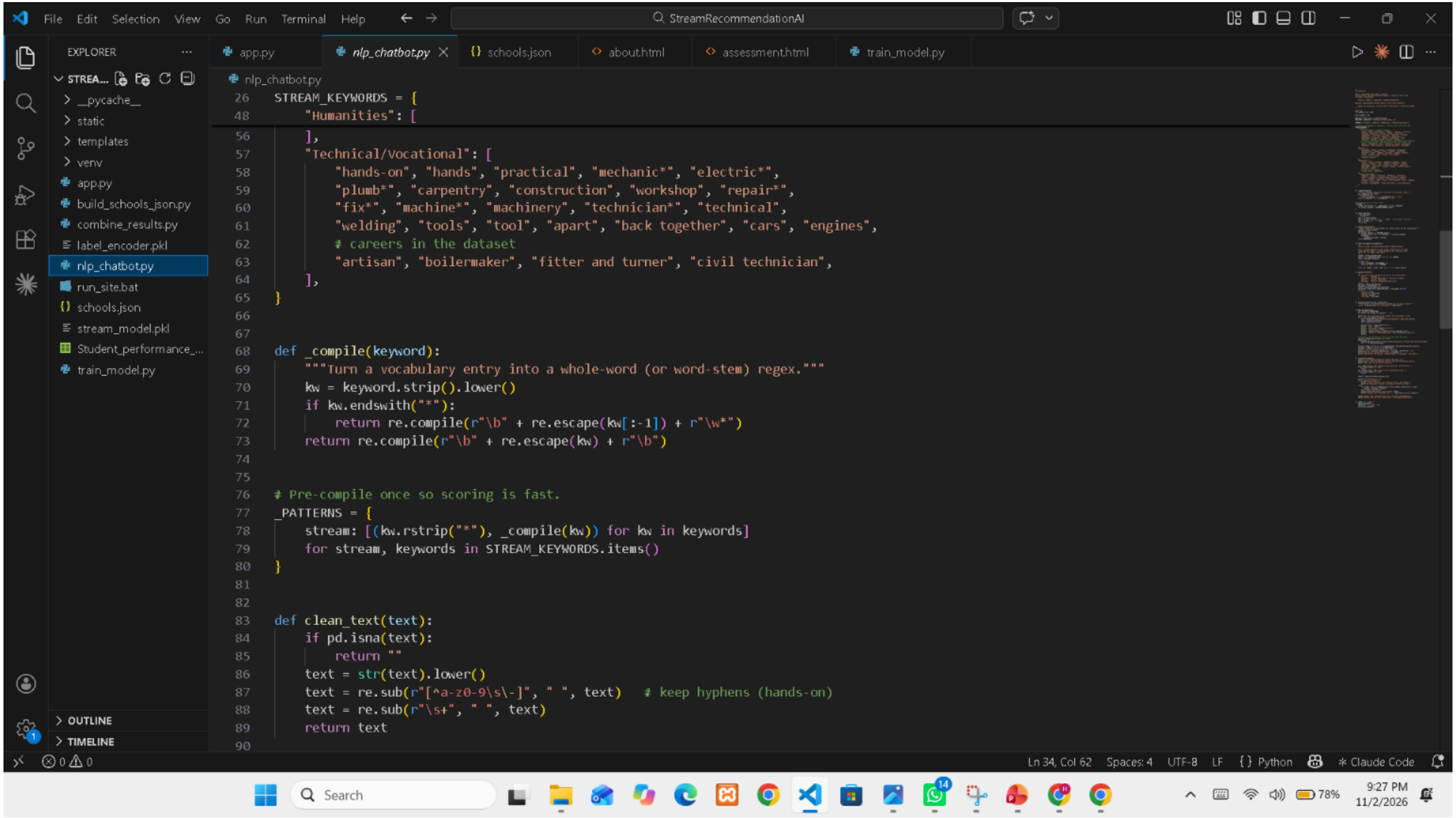

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
combine_results.py
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def recommend(marks, text, model, encoder, verbose=True):
"""
marks : {feature column: mark 0-100} for the 9 subjects
text : the learner's combined chatbot answers
Returns a plain dict (safe to store in a Flask session).
"""
marks = {c: float(marks[c]) for c in FEATURE_COLS}
ml = ml_scores(marks, model, encoder)
nlp = analyse_text(text)
if nlp["has_signal"]:
ml_w, nlp_w = ML_WEIGHT, NLP_WEIGHT
else:
ml_w, nlp_w = 1.0, 0.0 # no interest keywords found -> marks only
final = {s: round(ml_w * ml[s] + nlp_w * nlp["scores"][s], 1) for s in STREAMS}
ranking = sorted(final.items(), key=lambda item: -item[1])
best_stream, best_score = ranking[0]
runner_up, runner_up_score = ranking[1]
matched = nlp["matches"].get(best_stream, [])
reason = build_reason(best_stream, marks, matched, nlp["has_signal"])
result = {
"recommended_stream": best_stream,
"confidence": best_score,
"fit": best_score, # same number, extra names so
"fit_percentage": best_score, # older templates keep working
"reason": reason,
"careers": STREAM_CAREERS[best_stream],
"possible_careers": STREAM_CAREERS[best_stream],
"runner_up": runner_up,
"runner_up_score": runner_up_score,
list of {stream, score}, best first - Flask sessions do not keep dict order
"ranking": [{"stream": s, "score": v} for s, v in ranking],
"final_scores": final,
Ln 219, Col 36 Spaces: 4 UTF-8 LF Python Claude Code
Search
80%
9:24 PM
11/2/2026

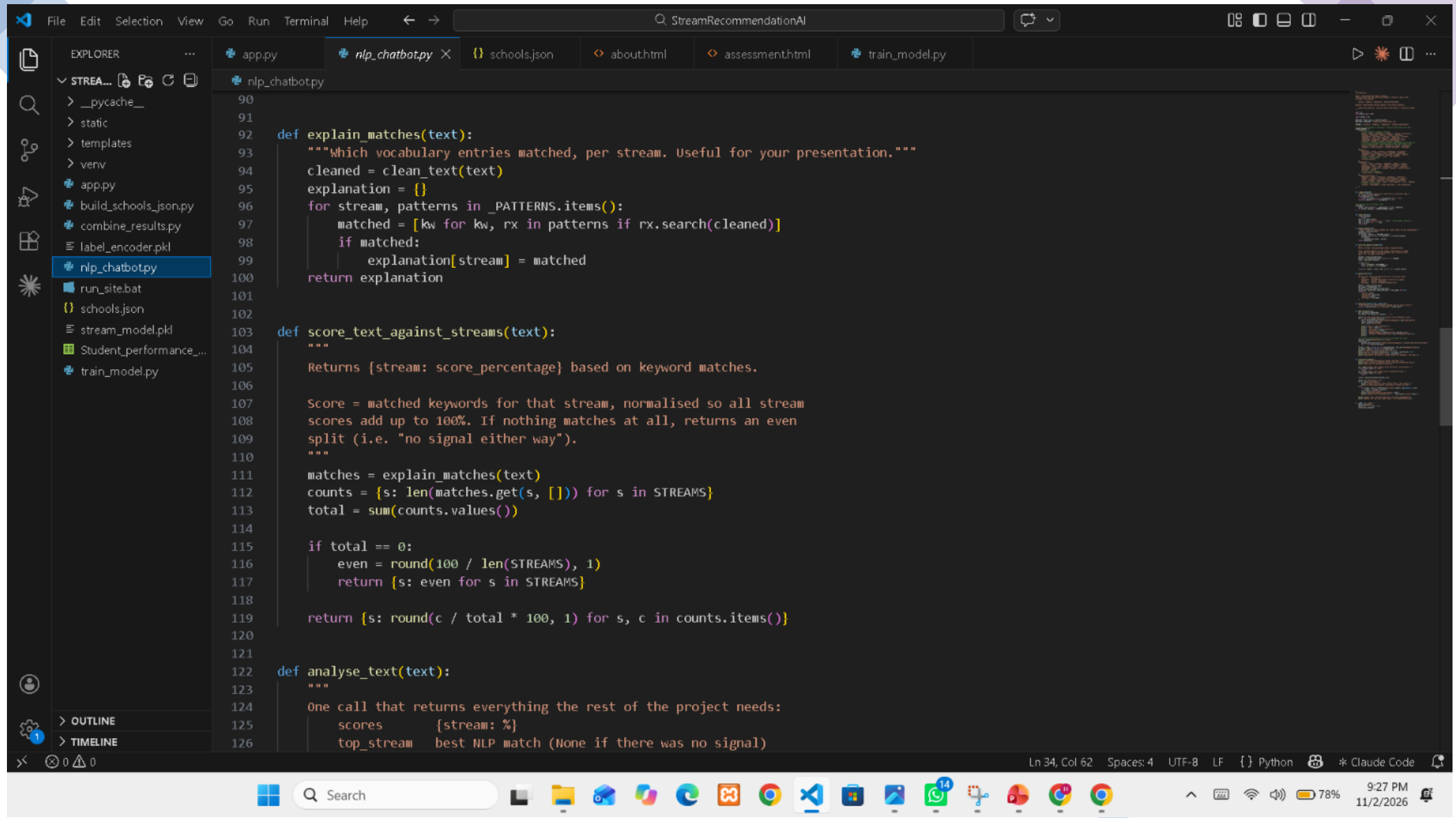

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
combine_results.py
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def recommend(marks, text, model, encoder, verbose=True):
if verbose:
print("\n=== Final Recommendation ===")
print(f"Recommended Stream: {best_stream}")
print(f"Fit: {best_score}%")
print(f"Reason: {reason}")
print(f"Possible careers: {', '.join(STREAM_CAREERS[best_stream])}")
print(f"\n(ML scores: {ml})")
print(f"(NLP scores: {nlp['scores']})")
print(f"(Combined scores: {final})")
print(f"(Weights used: ML {ml_w:.0%} / NLP {nlp_w:.0%})")
return result
def validate(model, encoder):
"""ML-only vs NLP-only vs Combined accuracy on the unseen Test learners."""
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.replace(" ", "_")
test_df = df[df["Dataset_Split"] == "Test"]
actual = list(test_df[TARGET_COL])
n = len(test_df)
ml_pred = list(encoder.inverse_transform(model.predict(test_df[FEATURE_COLS])))
nlp_pred, combined_pred = [], []
for _, row in test_df.iterrows():
text = build_text(row["Chatbot_Interest_Response"], row["Career_Goal"])
nlp_pred.append(analyse_text(text)["top_stream"])
marks = {c: row[c] for c in FEATURE_COLS}
combined_pred.append(recommend(marks, text, model, encoder, verbose=False)["recommended_stream"])
def correct(pred):
return sum(p == a for p, a in zip(pred, actual))
print("=== Validation: ML-only vs NLP-only vs Combined ===")
Ln 219, Col 36 Spaces: 4 UTF-8 LF Python Claude Code
Search
79%
9:25 PM
11/2/2026

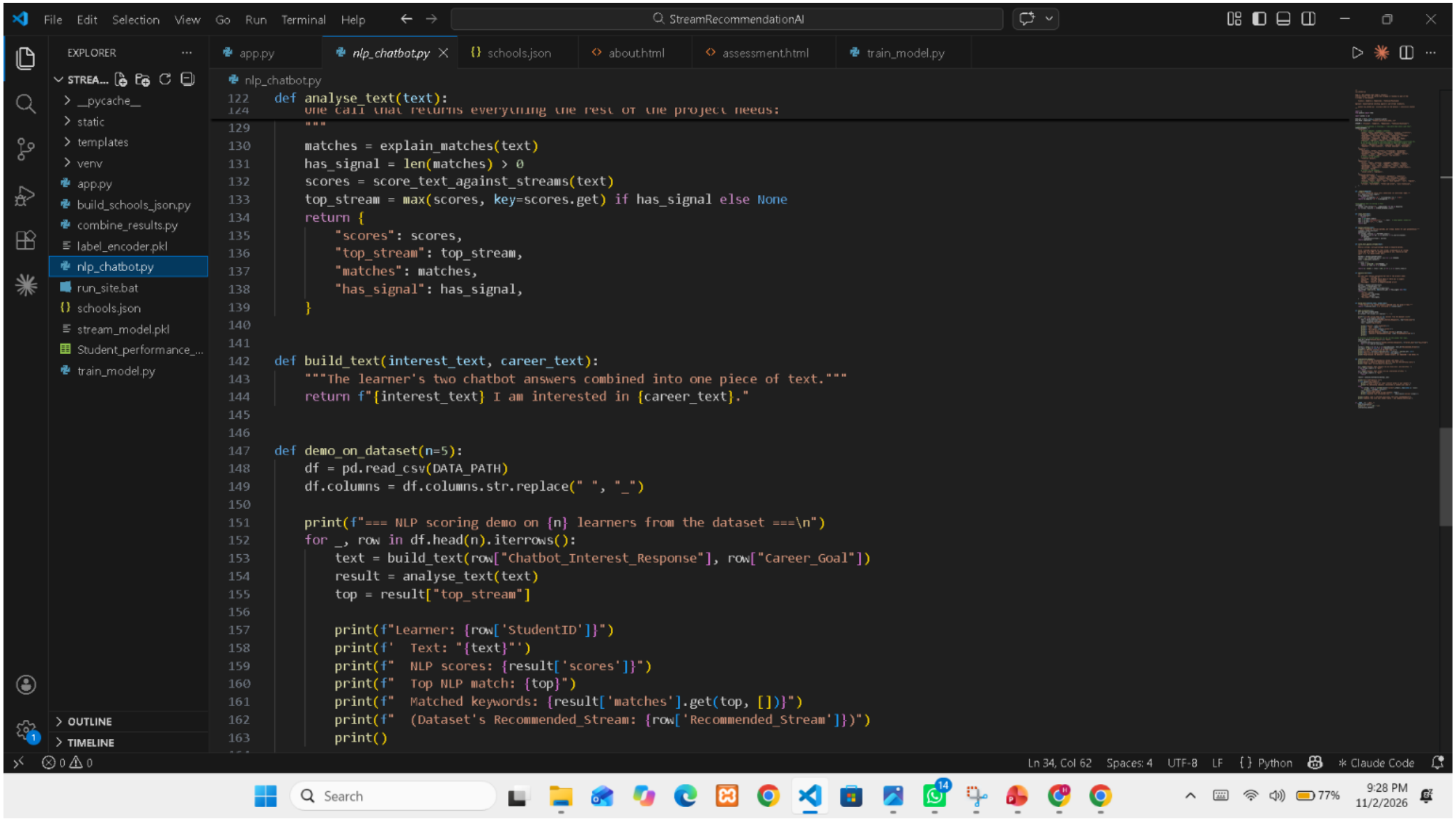

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
combine_results.py
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def _ask_mark(label):
while True:
try:
if 0 <= value <= 100:
except ValueError:
pass
print(" Please enter a number from 0 to 100.")
def try_it_yourself(model, encoder):
print("=== Try it yourself ===")
marks = {c: _ask_mark(SUBJECT_LABELS[c]) for c in FEATURE_COLS}
interest = input("What subjects do you enjoy most, and why?\nYou: ")
career = input("What career are you interested in?\nYou: ")
recommend(marks, build_text(interest, career), model, encoder, verbose=True)
if __name__ == "__main__":
model, encoder = load_model()
validate(model, encoder)
print("\n" + "=" * 60 + "\n")
print("=== Worked example ===")
example_marks = {
"Mathematics": 88, "Life_Orientation": 70, "First_Additional_Language": 60,
"Social_Science": 55, "EMS": 58, "Home_Language": 65,
"Creative_Arts": 62, "Natural_Science": 85, "Technology": 78,
}
example_text = "I enjoy Maths and Science because I like solving problems and learning how things work."
print(f"Marks: {example_marks}")
print(f'Interest text: "{example_text}"')
recommend(example_marks, example_text, model, encoder, verbose=True)
print("\n" + "=" * 60 + "\n")
try_it_yourself(model, encoder)
Ln 219, Col 36 Spaces: 4 UTF-8 LF Python Claude Code
Search
79%
9:25 PM
11/2/2026

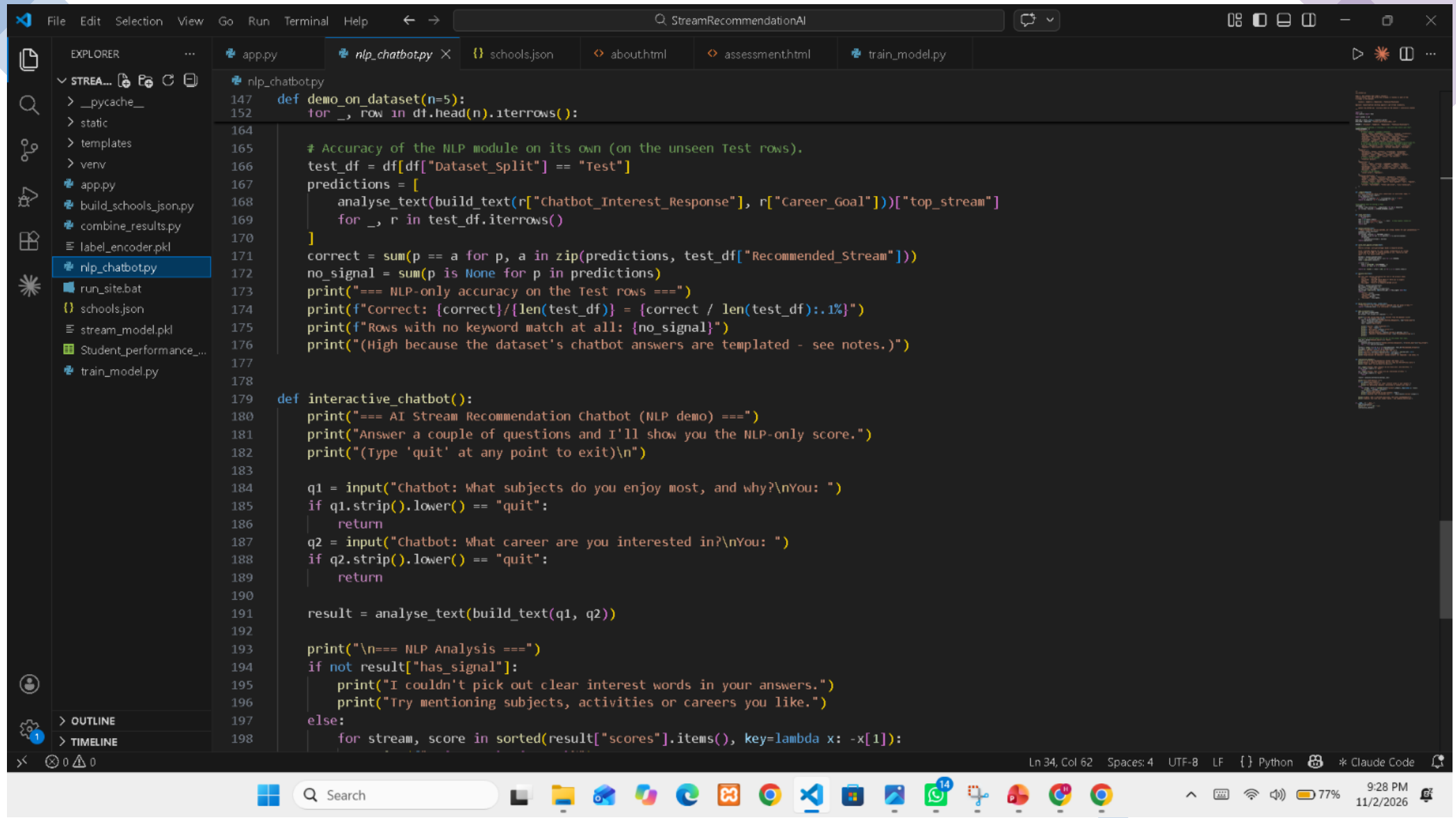


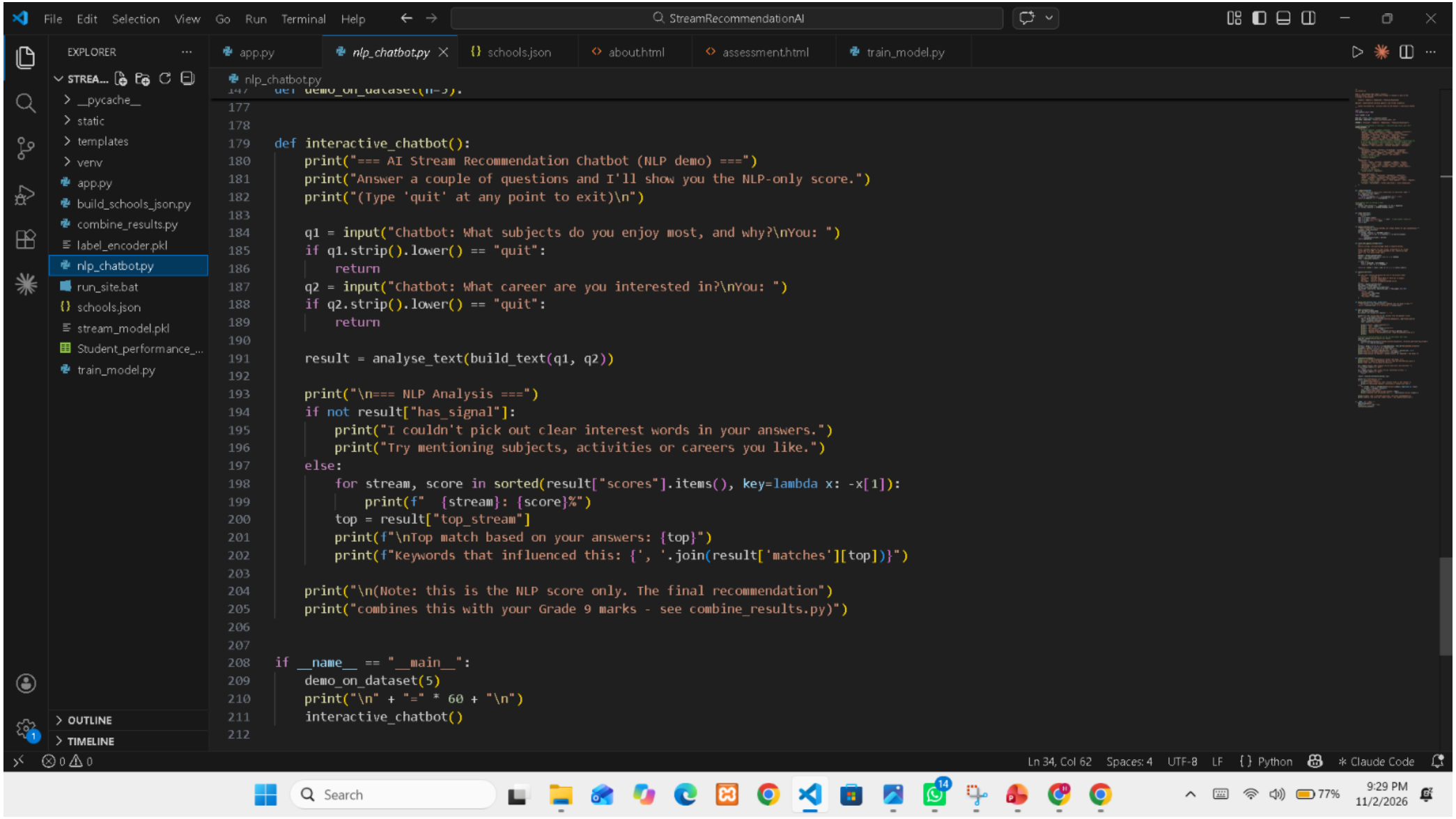
nlp_chatbot.py

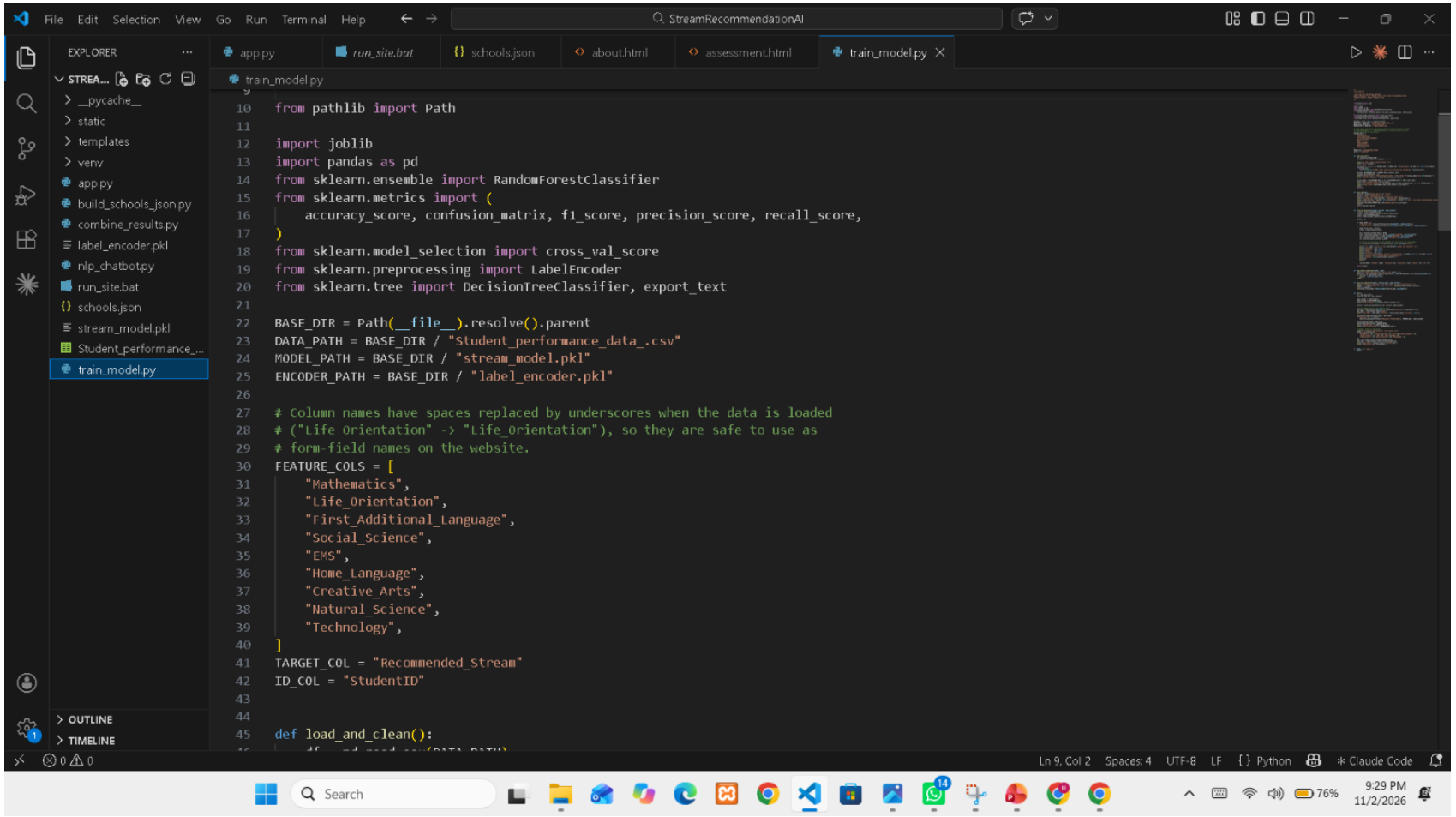

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
nlp_chatbot.py
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
STREAM_KEYWORDS = {
"Humanities": [
],
"Technical/Vocational": [
"hands-on", "hands", "practical", "mechanic*", "electric*",
"plumb*", "carpentry", "construction", "workshop", "repair*",
"fix*", "machine*", "machinery", "technician*", "technical",
"welding", "tools", "tool", "apart", "back together", "cars", "engines",
careers in the dataset
"artisan", "boilermaker", "fitter and turner", "civil technician",
],
}
def _compile(keyword):
"""Turn a vocabulary entry into a whole-word (or word-stem) regex."""
kw = keyword.strip().lower()
if kw.endswith("*"):
return re.compile(r"\b" + re.escape(kw[:-1]) + r"\w*")
return re.compile(r"\b" + re.escape(kw) + r"\b")
Pre-compile once so scoring is fast.
_PATTERNS = {
stream: [(kw.rstrip("*"), _compile(kw)) for kw in keywords]
for stream, keywords in STREAM_KEYWORDS.items()
}
def clean_text(text):
if pd.isna(text):
return ""
text = str(text).lower()
text = re.sub(r"[^a-z0-9\s\-]", " ", text) # keep hyphens (hands-on)
text = re.sub(r"\s+", " ", text)
return text
Ln 34, Col 62
Spaces: 4
UTF-8
LF
Python
Claude Code
Search
78%
9:27 PM
11/2/2026


File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
nlp_chatbot.py
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def explain_matches(text):
"""Which vocabulary entries matched, per stream. Useful for your presentation."""
cleaned = clean_text(text)
explanation = {}
for stream, patterns in _PATTERNS.items():
matched = [kw for kw, rx in patterns if rx.search(cleaned)]
if matched:
explanation[stream] = matched
return explanation
def score_text_against_streams(text):
"""
Returns {stream: score_percentage} based on keyword matches.
Score = matched keywords for that stream, normalised so all stream
scores add up to 100%. If nothing matches at all, returns an even
split (i.e. "no signal either way").
"""
matches = explain_matches(text)
counts = {s: len(matches.get(s, [])) for s in STREAMS}
total = sum(counts.values())
if total == 0:
even = round(100 / len(STREAMS), 1)
return {s: even for s in STREAMS}
return {s: round(c / total * 100, 1) for s, c in counts.items()}
def analyse_text(text):
"""
One call that returns everything the rest of the project needs:
scores {stream: %}
top_stream best NLP match (None if there was no signal)
Ln 34, Col 62 Spaces: 4 UTF-8 LF Python Claude Code
Search
78%
9:27 PM
11/2/2026

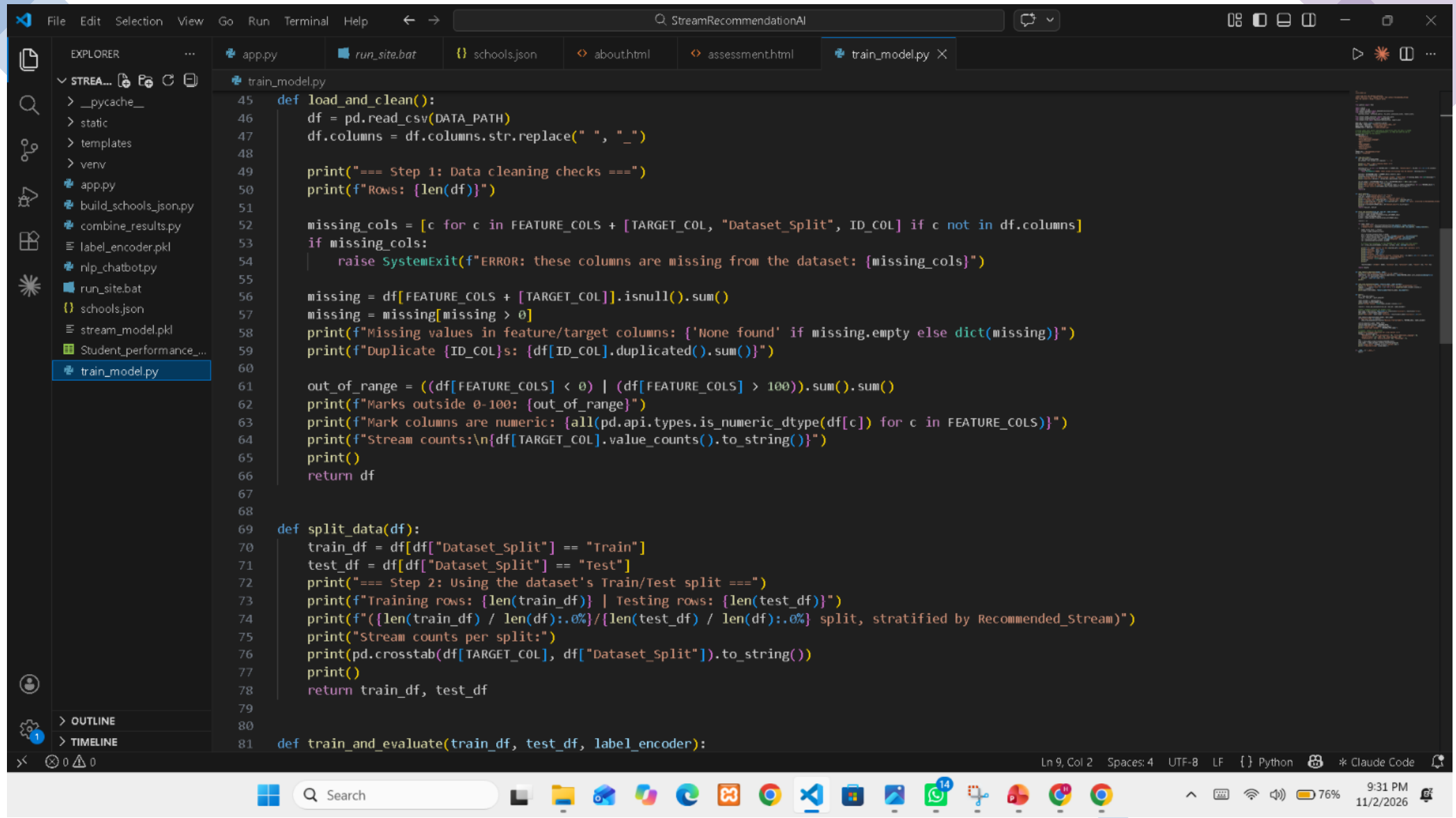

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
nlp_chatbot.py
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def analyse_text(text):
"""
matches = explain_matches(text)
has_signal = len(matches) > 0
scores = score_text_against_streams(text)
top_stream = max(scores, key=scores.get) if has_signal else None
return {
"scores": scores,
"top_stream": top_stream,
"matches": matches,
"has_signal": has_signal,
}
def build_text(interest_text, career_text):
"""The learner's two chatbot answers combined into one piece of text."""
return f"{interest_text} I am interested in {career_text}."
def demo_on_dataset(n=5):
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.replace(" ", "_")
print(f"=== NLP scoring demo on {n} learners from the dataset ===\n")
for _, row in df.head(n).iterrows():
text = build_text(row["Chatbot_Interest_Response"], row["Career_Goal"])
result = analyse_text(text)
top = result["top_stream"]
print(f"Learner: {row['StudentID']}")
print(f' Text: "{text}"')
print(f" NLP scores: {result['scores']}")
print(f" Top NLP match: {top}")
print(f" Matched keywords: {result['matches'].get(top, [])}")
print(f" (Dataset's Recommended_Stream: {row['Recommended_Stream']})")
print()
Ln 34, Col 62
Spaces: 4
UTF-8
LF
Python
Claude Code
Search
77%
9:28 PM
11/2/2026

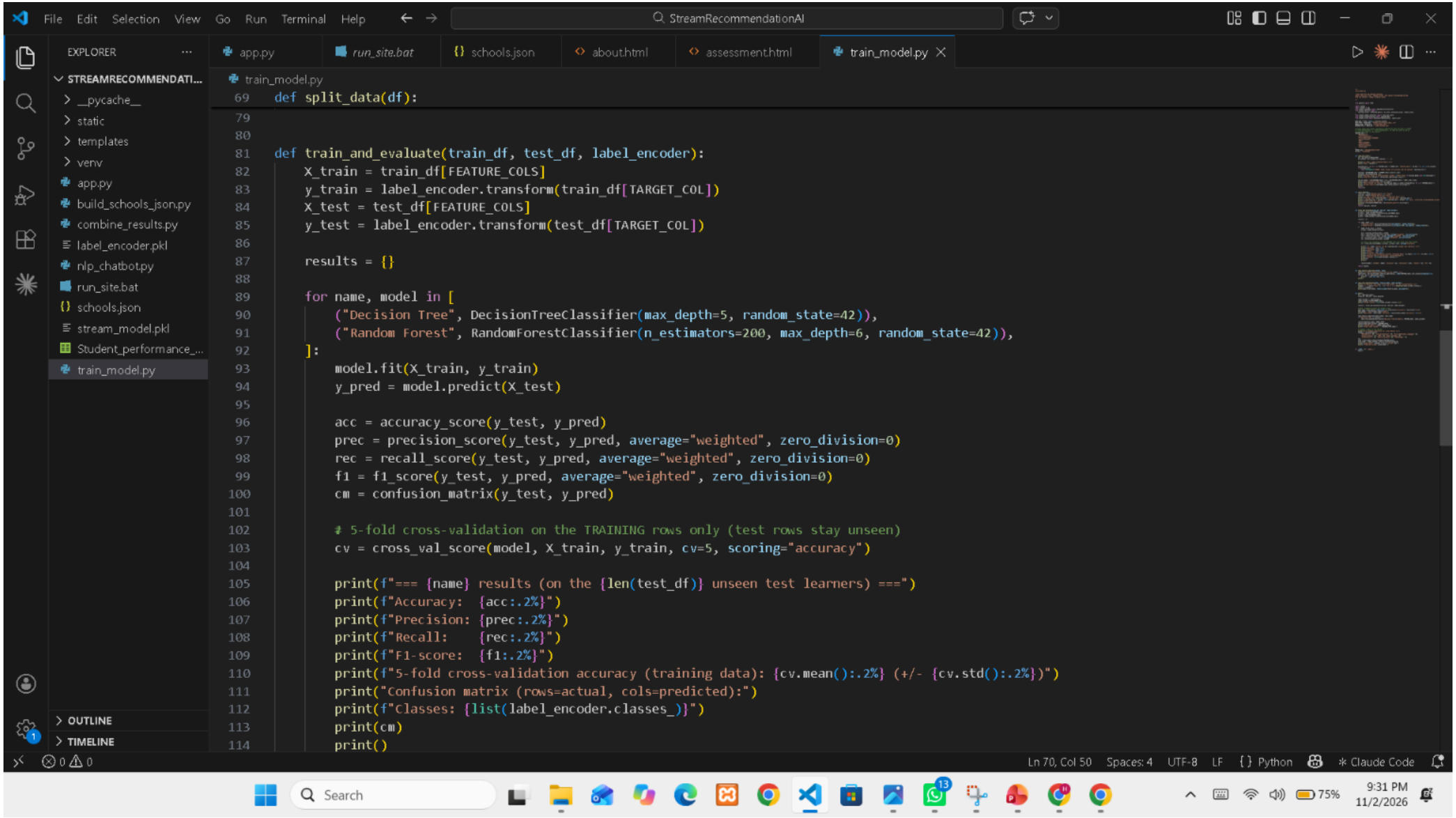

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
nlp_chatbot.py
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def demo_on_dataset(n=5):
for _, row in df.head(n).iterrows():
Accuracy of the NLP module on its own (on the unseen Test rows).
test_df = df[df["Dataset_Split"] == "Test"]
predictions = [
analyse_text(build_text(r["Chatbot_Interest_Response"], r["Career_Goal"]))["top_stream"]
for _, r in test_df.iterrows()
]
correct = sum(p == a for p, a in zip(predictions, test_df["Recommended_Stream"]))
no_signal = sum(p is None for p in predictions)
print("=== NLP-only accuracy on the Test rows ===")
print(f"Correct: {correct}/{len(test_df)} = {correct / len(test_df):.1%}")
print(f"Rows with no keyword match at all: {no_signal}")
print("(High because the dataset's chatbot answers are templated - see notes.)")
def interactive_chatbot():
print("=== AI Stream Recommendation Chatbot (NLP demo) ===")
print("Answer a couple of questions and I'll show you the NLP-only score.")
print("(Type 'quit' at any point to exit)\n")
q1 = input("Chatbot: What subjects do you enjoy most, and why?\nYou: ")
if q1.strip().lower() == "quit":
return
q2 = input("Chatbot: What career are you interested in?\nYou: ")
if q2.strip().lower() == "quit":
return
result = analyse_text(build_text(q1, q2))
print("\n=== NLP Analysis ===")
if not result["has_signal"]:
print("I couldn't pick out clear interest words in your answers.")
print("Try mentioning subjects, activities or careers you like.")
else:
for stream, score in sorted(result["scores"].items(), key=lambda x: -x[1]):
Ln 34, Col 62 Spaces: 4 UTF-8 LF Python Claude Code
Search
77%
9:28 PM
11/2/2026

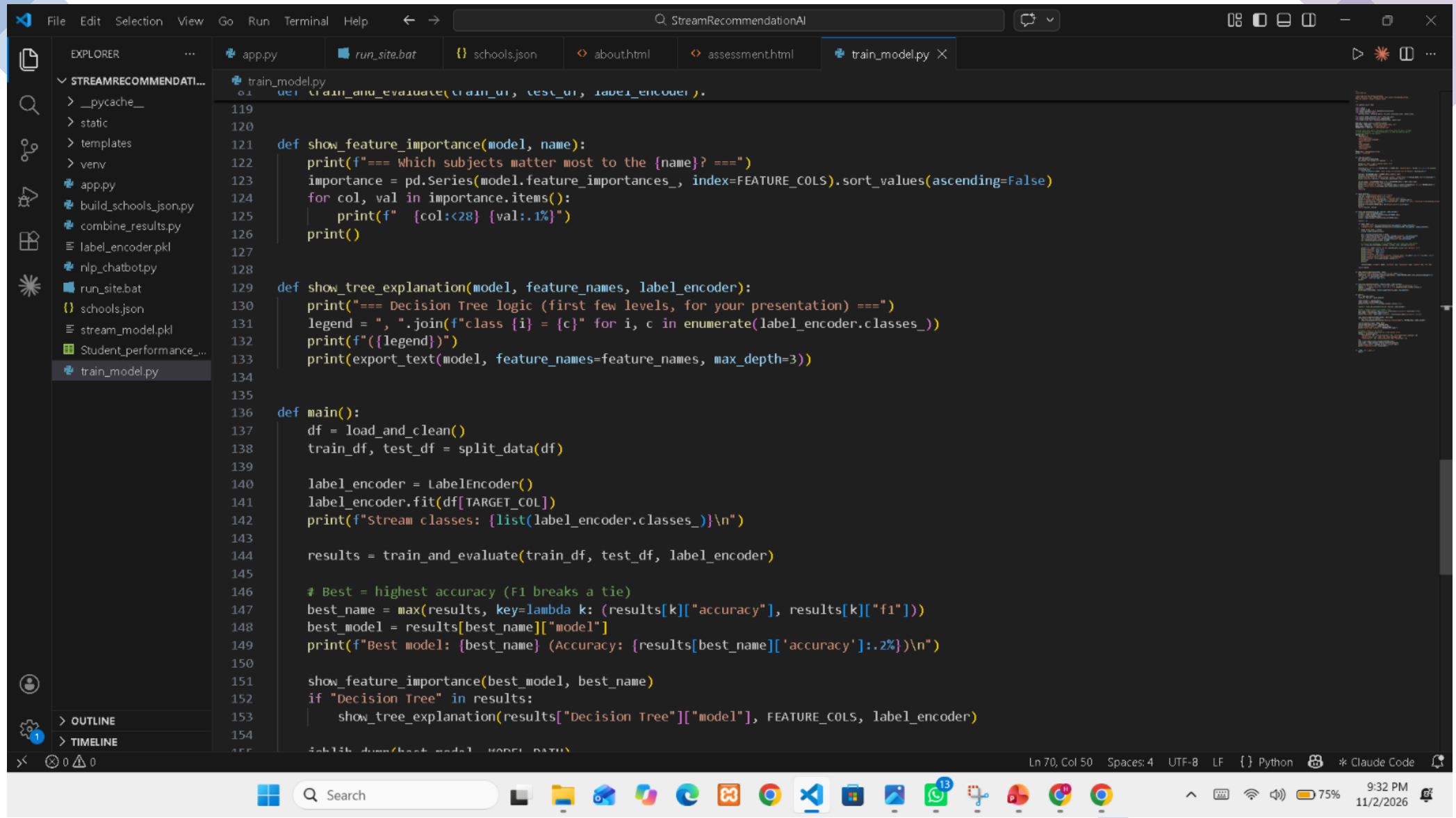

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
nlp_chatbot.py
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def interactive_chatbot():
print("=== AI Stream Recommendation Chatbot (NLP demo) ===")
print("Answer a couple of questions and I'll show you the NLP-only score.")
print("(Type 'quit' at any point to exit)\n")
q1 = input("Chatbot: What subjects do you enjoy most, and why?\nYou: ")
if q1.strip().lower() == "quit":
return
q2 = input("Chatbot: What career are you interested in?\nYou: ")
if q2.strip().lower() == "quit":
return
result = analyse_text(build_text(q1, q2))
print("\n=== NLP Analysis ===")
if not result["has_signal"]:
print("I couldn't pick out clear interest words in your answers.")
print("Try mentioning subjects, activities or careers you like.")
else:
for stream, score in sorted(result["scores"].items(), key=lambda x: -x[1]):
print(f" {stream}: {score}%")
top = result["top_stream"]
print(f"\nTop match based on your answers: {top}")
print(f"Keywords that influenced this: {', '.join(result['matches'][top])}")
print("\n(Note: this is the NLP score only. The final recommendation")
print("combines this with your Grade 9 marks - see combine_results.py)")
if __name__ == "__main__":
demo_on_dataset(5)
print("\n" + "=" * 60 + "\n")
interactive_chatbot()
Ln 34, Col 62
Spaces: 4
UTF-8
LF
Python
Claude Code
Search
77%
9:29 PM
11/2/2026

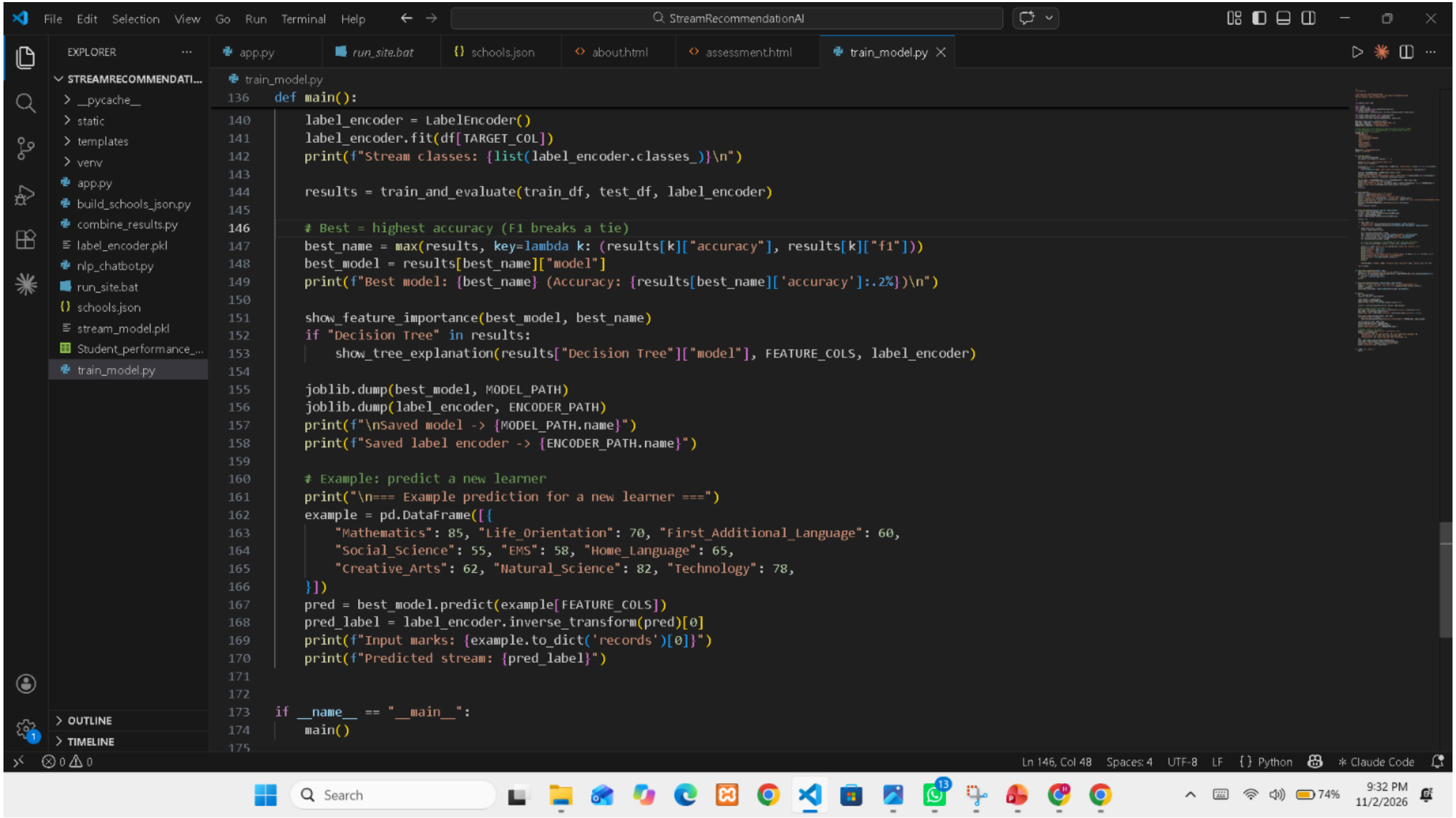

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
from pathlib import Path
import joblib
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
accuracy_score, confusion_matrix, f1_score, precision_score, recall_score,
)
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, export_text
BASE_DIR = Path(__file__).resolve().parent
DATA_PATH = BASE_DIR / "Student_performance_data_.csv"
MODEL_PATH = BASE_DIR / "stream_model.pkl"
ENCODER_PATH = BASE_DIR / "label_encoder.pkl"
Column names have spaces replaced by underscores when the data is loaded
("Life Orientation" -> "Life_Orientation"), so they are safe to use as
form-field names on the website.
FEATURE_COLS = [
"Mathematics",
"Life_Orientation",
"First_Additional_Language",
"Social_Science",
"EMS",
"Home_Language",
"Creative_Arts",
"Natural_Science",
"Technology",
]
TARGET_COL = "Recommended_Stream"
ID_COL = "StudentID"
def load_and_clean():
Ln 9, Col 2 Spaces: 4 UTF-8 LF Python Claude Code
Search
76%
9:29 PM
11/2/2026

train_model


File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
def load_and_clean():
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.replace(" ", "_")
print("=== Step 1: Data cleaning checks ===")
print(f"Rows: {len(df)}")
missing_cols = [c for c in FEATURE_COLS + [TARGET_COL, "Dataset_Split", ID_COL] if c not in df.columns]
if missing_cols:
raise SystemExit(f"ERROR: these columns are missing from the dataset: {missing_cols}")
missing = df[FEATURE_COLS + [TARGET_COL]].isnull().sum()
missing = missing[missing > 0]
print(f"Missing values in feature/target columns: {'None found' if missing.empty else dict(missing)}")
print(f"Duplicate {ID_COL}s: {df[ID_COL].duplicated().sum()}")
out_of_range = ((df[FEATURE_COLS] < 0) | (df[FEATURE_COLS] > 100)).sum().sum()
print(f"Marks outside 0-100: {out_of_range}")
print(f"Mark columns are numeric: {all(pd.api.types.is_numeric_dtype(df[c]) for c in FEATURE_COLS)}")
print(f"Stream counts:\n{df[TARGET_COL].value_counts().to_string()}")
print()
return df
def split_data(df):
train_df = df[df["Dataset_Split"] == "Train"]
test_df = df[df["Dataset_Split"] == "Test"]
print("=== Step 2: Using the dataset's Train/Test split ===")
print(f"Training rows: {len(train_df)} | Testing rows: {len(test_df)}")
print(f"({len(train_df) / len(df):.0%}/{len(test_df) / len(df):.0%} split, stratified by Recommended_Stream)")
print("Stream counts per split:")
print(pd.crosstab(df[TARGET_COL], df["Dataset_Split"]).to_string())
print()
return train_df, test_df
def train_and_evaluate(train_df, test_df, label_encoder):
Ln 9, Col 2 Spaces: 4 UTF-8 LF Python Claude Code
Search
76%
9:31 PM
11/2/2026

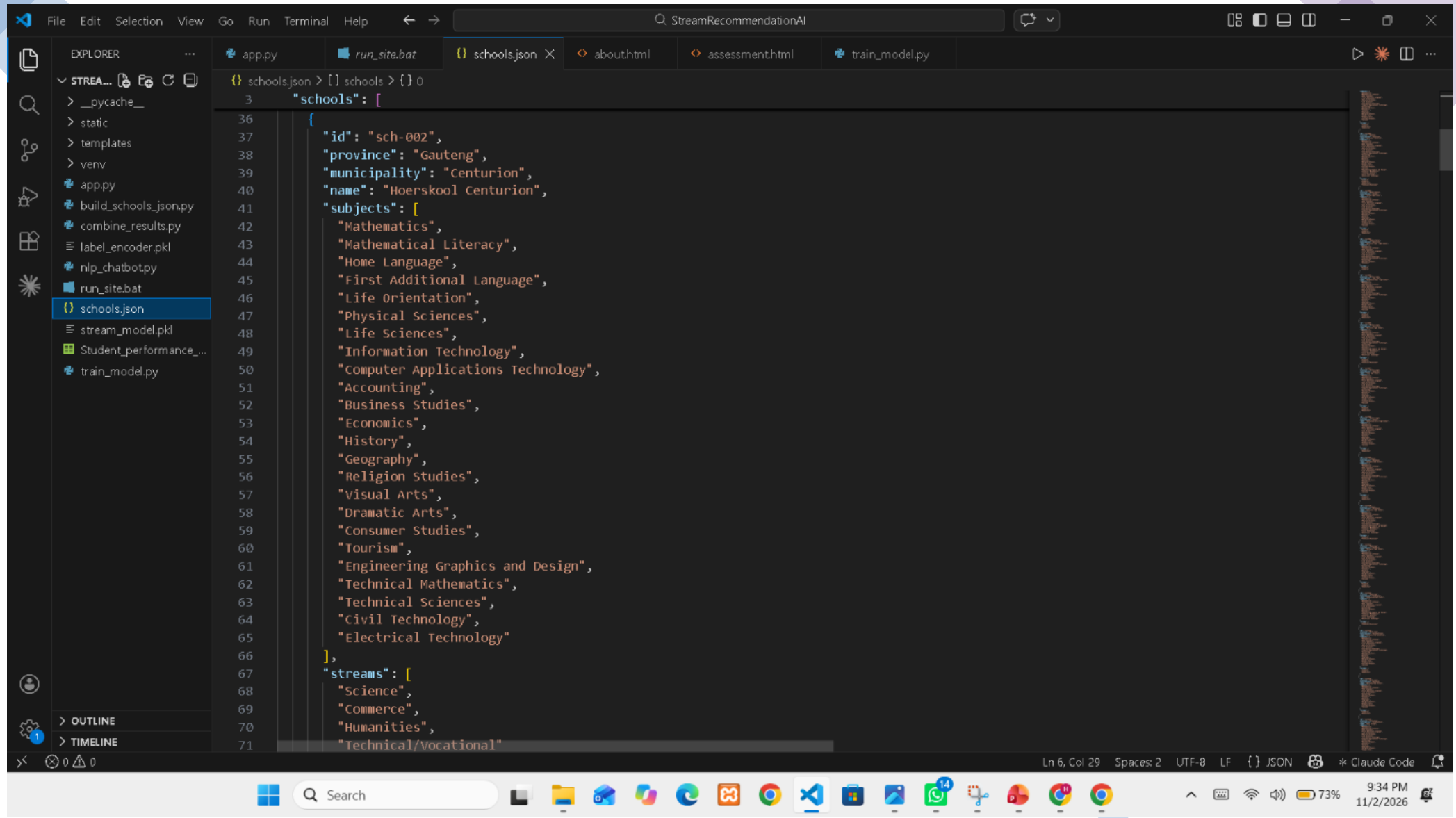

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
STREAMRECOMMENDATI...
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
train_model.py
def split_data(df):
def train_and_evaluate(train_df, test_df, label_encoder):
X_train = train_df[FEATURE_COLS]
y_train = label_encoder.transform(train_df[TARGET_COL])
X_test = test_df[FEATURE_COLS]
y_test = label_encoder.transform(test_df[TARGET_COL])
results = {}
for name, model in [
("Decision Tree", DecisionTreeClassifier(max_depth=5, random_state=42)),
("Random Forest", RandomForestClassifier(n_estimators=200, max_depth=6, random_state=42)),
]:
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average="weighted", zero_division=0)
rec = recall_score(y_test, y_pred, average="weighted", zero_division=0)
f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)
cm = confusion_matrix(y_test, y_pred)
5-fold cross-validation on the TRAINING rows only (test rows stay unseen)
cv = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
print(f"=== {name} results (on the {len(test_df)} unseen test learners) ===")
print(f"Accuracy: {acc:.2%}")
print(f"Precision: {prec:.2%}")
print(f"Recall: {rec:.2%}")
print(f"F1-score: {f1:.2%}")
print(f"5-fold cross-validation accuracy (training data): {cv.mean():.2%} (+/- {cv.std():.2%})")
print("Confusion matrix (rows=actual, cols=predicted):")
print(f"Classes: {list(label_encoder.classes_)}")
print(cm)
print()
Ln 70, Col 50 Spaces: 4 UTF-8 LF Python Claude Code
Search
75%
9:31 PM
11/2/2026

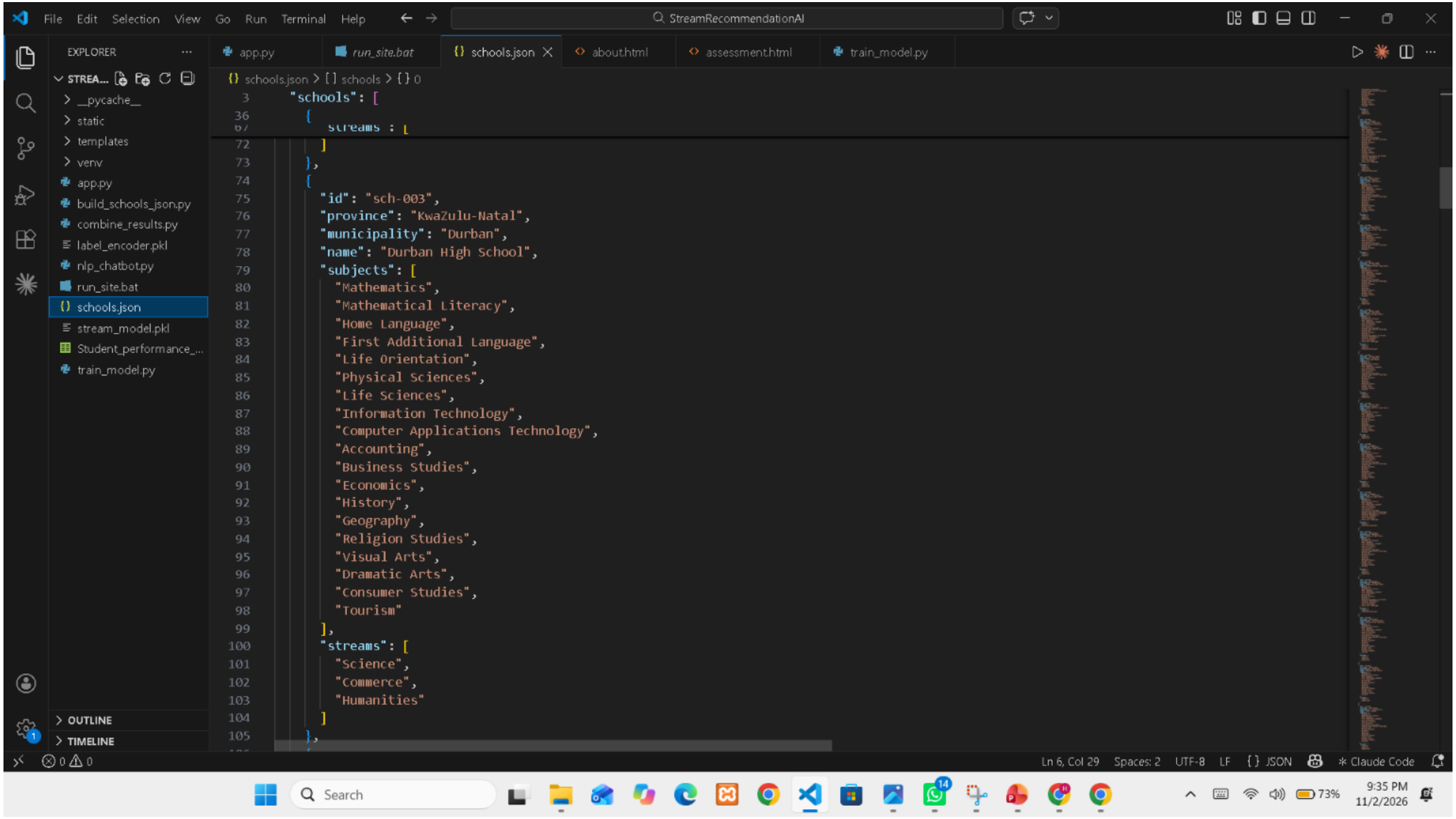

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
STREAMRECOMMENDATI...
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
train_model.py
def show_feature_importance(model, name):
print(f"=== Which subjects matter most to the {name}? ===")
importance = pd.Series(model.feature_importances_, index=FEATURE_COLS).sort_values(ascending=False)
for col, val in importance.items():
print(f" {col:<28} {val:.1%}")
print()
def show_tree_explanation(model, feature_names, label_encoder):
print("=== Decision Tree logic (first few levels, for your presentation) ===")
legend = ", ".join(f"class {i} = {c}" for i, c in enumerate(label_encoder.classes_))
print(f"({legend})")
print(export_text(model, feature_names=feature_names, max_depth=3))
def main():
df = load_and_clean()
train_df, test_df = split_data(df)
label_encoder = LabelEncoder()
label_encoder.fit(df[TARGET_COL])
print(f"Stream classes: {list(label_encoder.classes_)}\n")
results = train_and_evaluate(train_df, test_df, label_encoder)
Best = highest accuracy (F1 breaks a tie)
best_name = max(results, key=lambda k: (results[k]["accuracy"], results[k]["f1"]))
best_model = results[best_name]["model"]
print(f"Best model: {best_name} (Accuracy: {results[best_name]['accuracy']:.2%})\n")
show_feature_importance(best_model, best_name)
if "Decision Tree" in results:
show_tree_explanation(results["Decision Tree"]["model"], FEATURE_COLS, label_encoder)
OUTLINE
TIMELINE
Ln 70, Col 50 Spaces: 4 UTF-8 LF Python Claude Code
Search
75%
9:32 PM
11/2/2026

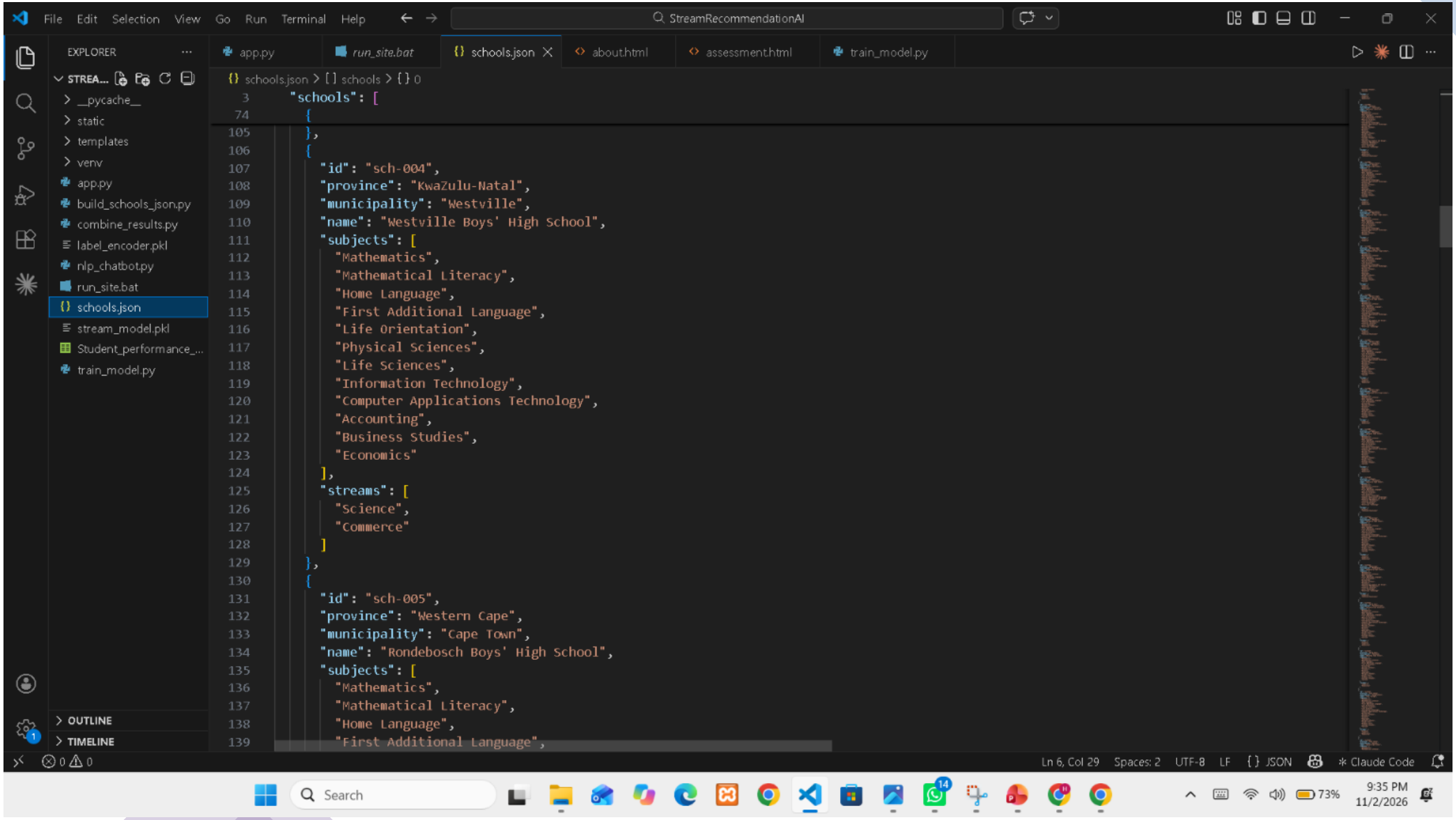

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
STREAMRECOMMENDATI...
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
136 def main():
140 label_encoder = LabelEncoder()
141 label_encoder.fit(df[TARGET_COL])
142 print(f"Stream classes: {list(label_encoder.classes_)}\n")
144 results = train_and_evaluate(train_df, test_df, label_encoder)
146 # Best = highest accuracy (F1 breaks a tie)
147 best_name = max(results, key=lambda k: (results[k]["accuracy"], results[k]["f1"]))
148 best_model = results[best_name]["model"]
149 print(f"Best model: {best_name} (Accuracy: {results[best_name]['accuracy']:.2%})\n")
151 show_feature_importance(best_model, best_name)
152 if "Decision Tree" in results:
153 show_tree_explanation(results["Decision Tree"]["model"], FEATURE_COLS, label_encoder)
155 joblib.dump(best_model, MODEL_PATH)
156 joblib.dump(label_encoder, ENCODER_PATH)
157 print(f"\nSaved model -> {MODEL_PATH.name}")
158 print(f"Saved label encoder -> {ENCODER_PATH.name}")
160 # Example: predict a new learner
161 print("\n=== Example prediction for a new learner ===")
162 example = pd.DataFrame([{
163 "Mathematics": 85, "Life_Orientation": 70, "First_Additional_Language": 60,
164 "Social_Science": 55, "EMS": 58, "Home_Language": 65,
165 "Creative_Arts": 62, "Natural_Science": 82, "Technology": 78,
166 }])
167 pred = best_model.predict(example[FEATURE_COLS])
168 pred_label = label_encoder.inverse_transform(pred)[0]
169 print(f"Input marks: {example.to_dict('records')[0]}")
170 print(f"Predicted stream: {pred_label}")
173 if __name__ == "__main__":
174 main()
Ln 146, Col 48 Spaces: 4 UTF-8 LF Python Claude Code
Search
74%
9:32 PM
11/2/2026

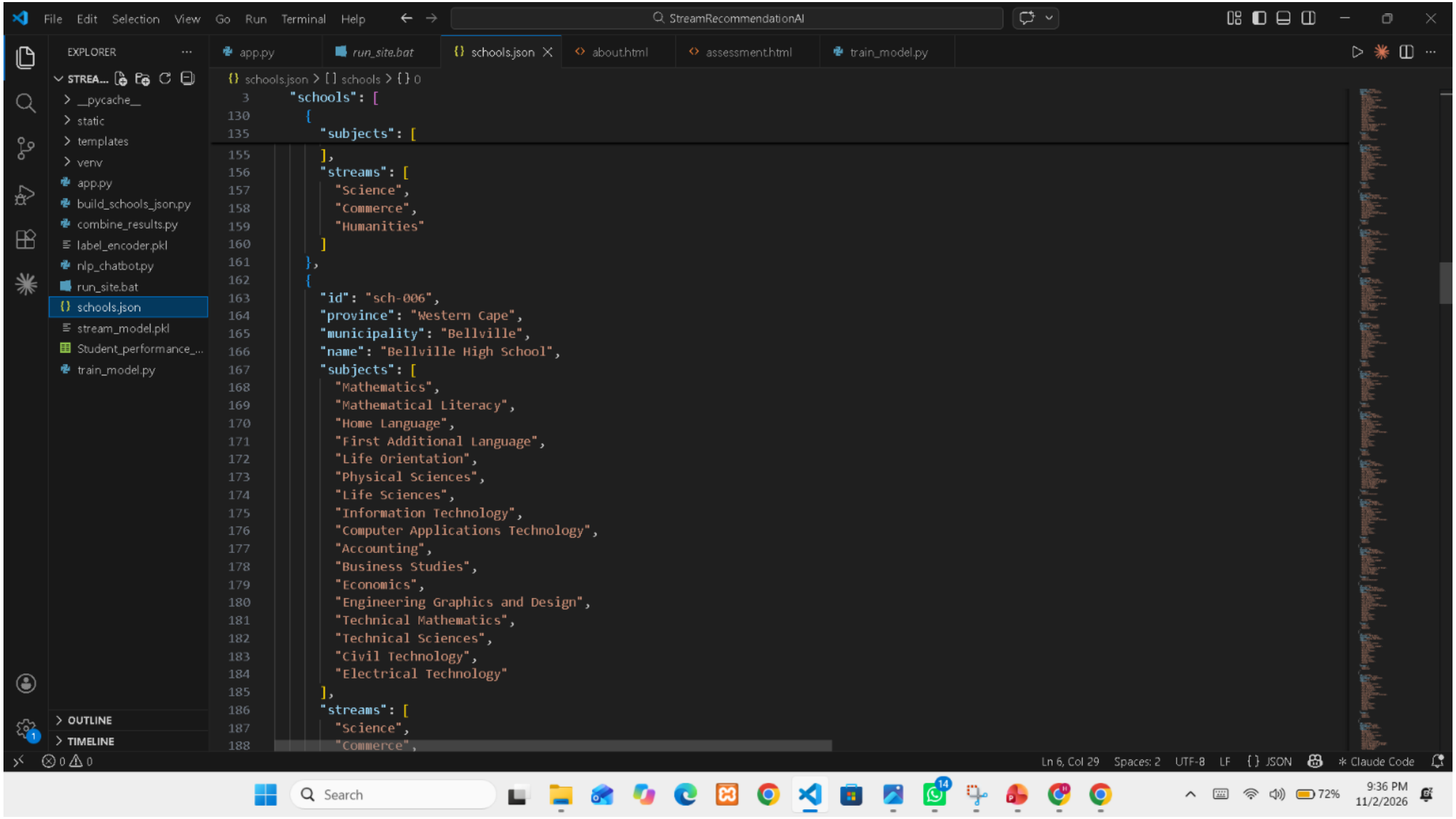

templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
"schools": [
{
"id": "sch-001",
"province": "Gauteng",
"municipality": "Johannesburg",
"name": "Parktown High School for Girls",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
"Life Orientation",
"Physical Sciences",
"Life Sciences",
"Information Technology",
"Computer Applications Technology",
"Accounting",
"Business Studies",
"Economics",
"History",
"Geography",
"Religion Studies",
"Visual Arts",
"Dramatic Arts",
"Consumer Studies",
"Tourism"
],
"streams": [
"Science",
"Commerce",
"Humanities"
]
},
{

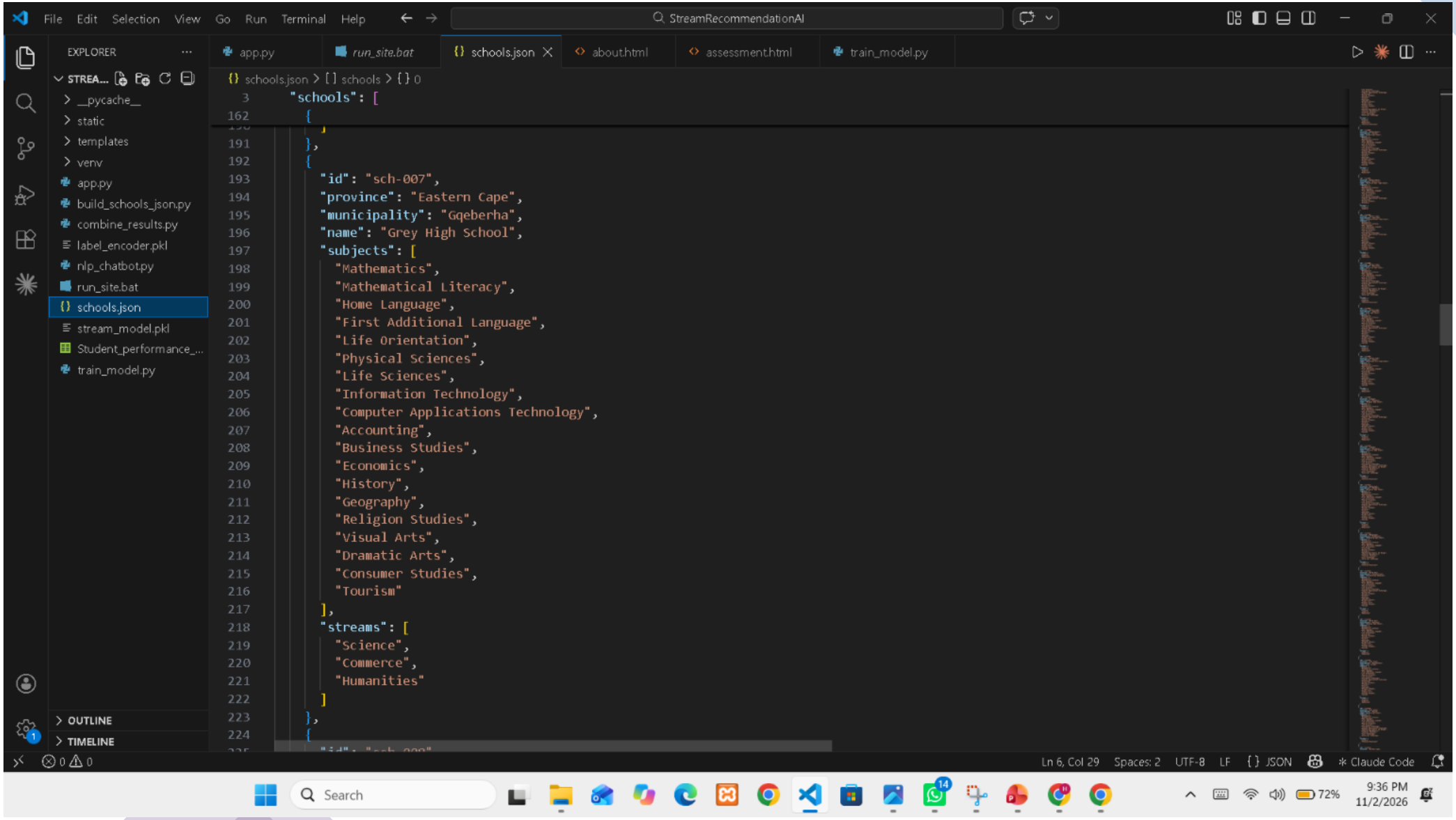
Schools.js

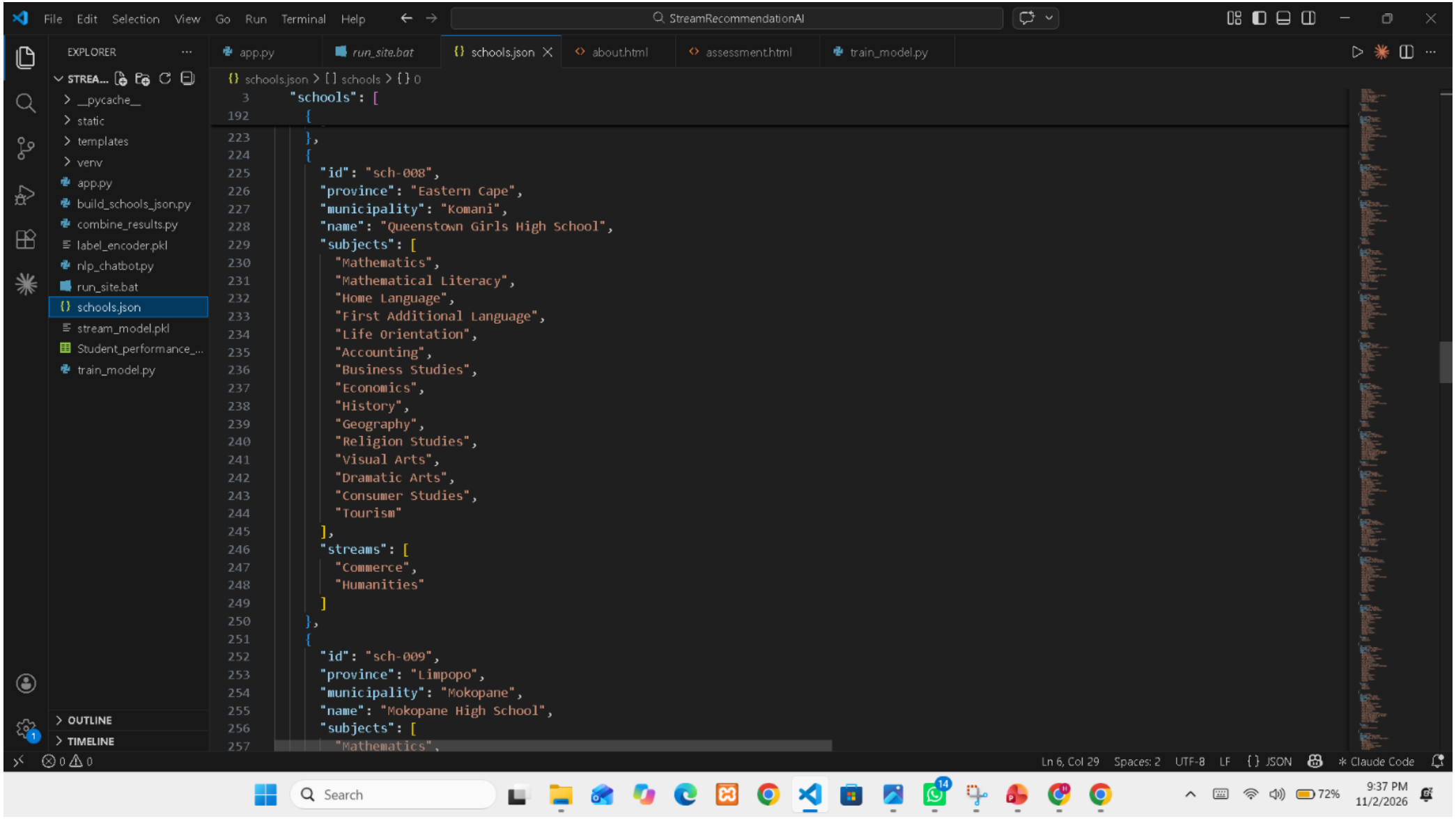

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
schools.json > [] schools > {} 0
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
"schools": [
{
"id": "sch-002",
"province": "Gauteng",
"municipality": "Centurion",
"name": "Hoerskool Centurion",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
"Life Orientation",
"Physical Sciences",
"Life Sciences",
"Information Technology",
"Computer Applications Technology",
"Accounting",
"Business Studies",
"Economics",
"History",
"Geography",
"Religion Studies",
"Visual Arts",
"Dramatic Arts",
"Consumer Studies",
"Tourism",
"Engineering Graphics and Design",
"Technical Mathematics",
"Technical Sciences",
"Civil Technology",
"Electrical Technology"
],
"streams": [
"Science",
"Commerce",
"Humanities",
"Technical/Vocational"
Ln 6, Col 29 Spaces: 2 UTF-8 LF { } JSON Claude Code
Search
73%
9:34 PM
11/2/2026

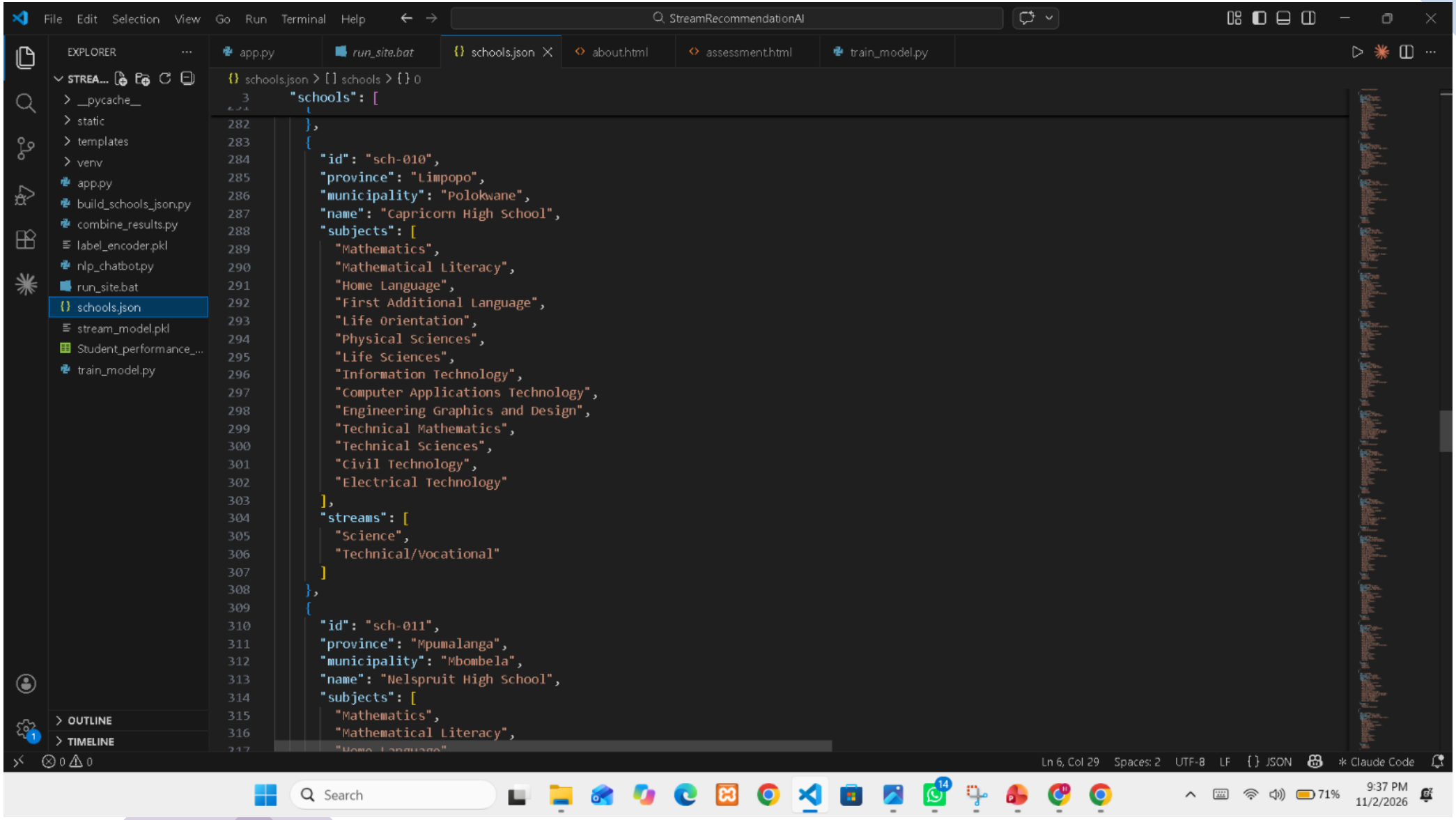

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
schools.json > [] schools > {} 0
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
"schools": [
"streams" : [
]
},
{
"id": "sch-003",
"province": "KwaZulu-Natal",
"municipality": "Durban",
"name": "Durban High School",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
"Life Orientation",
"Physical Sciences",
"Life Sciences",
"Information Technology",
"Computer Applications Technology",
"Accounting",
"Business Studies",
"Economics",
"History",
"Geography",
"Religion Studies",
"Visual Arts",
"Dramatic Arts",
"Consumer Studies",
"Tourism"
],
"streams": [
"Science",
"Commerce",
"Humanities"
]
},
Ln 6, Col 29 Spaces: 2 UTF-8 LF { } JSON Claude Code
Search
73%
9:35 PM
11/2/2026

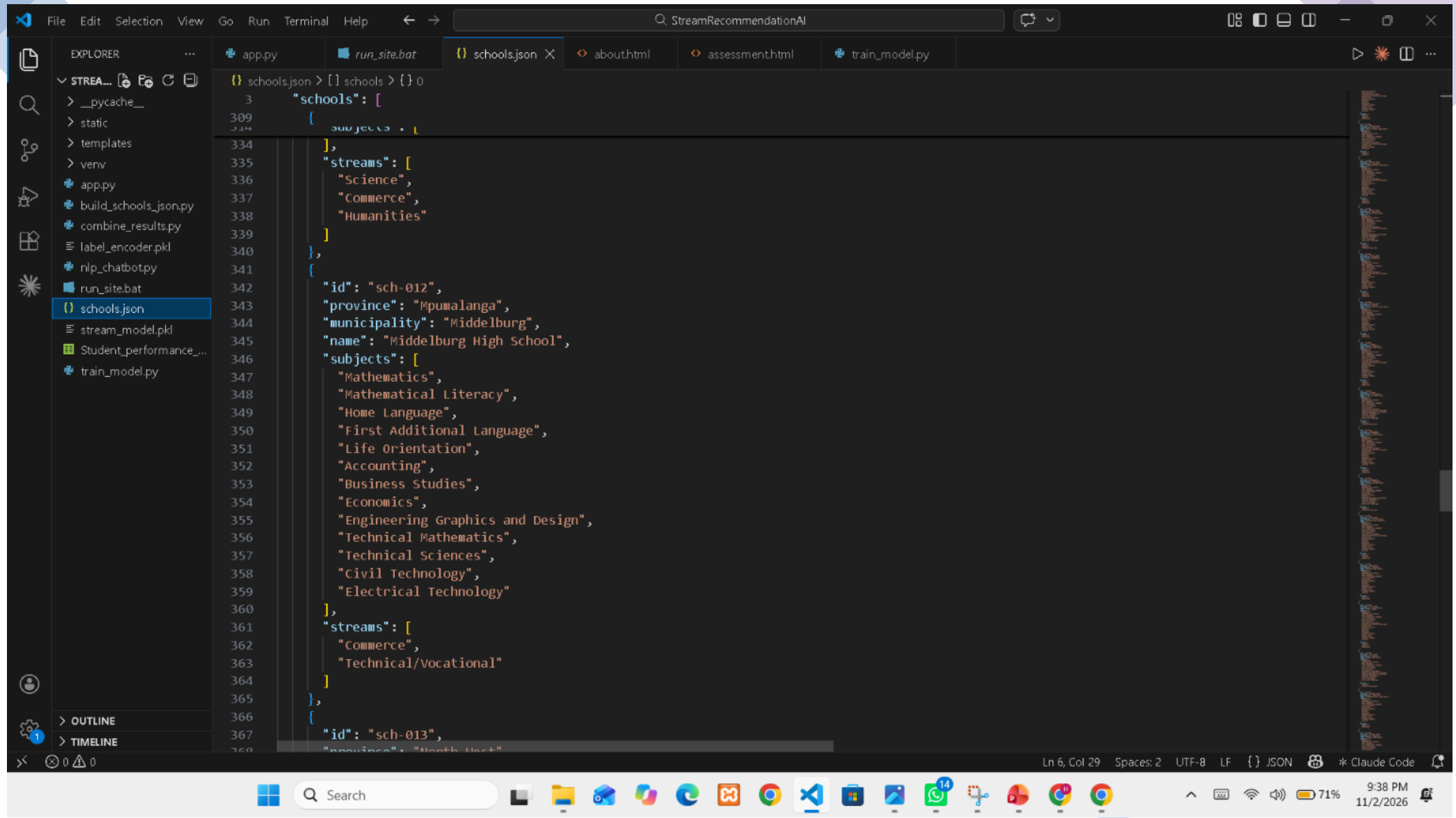

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
schools.json > [] schools > {} 0
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
"schools": [
{
},
{
"id": "sch-004",
"province": "KwaZulu-Natal",
"municipality": "Westville",
"name": "Westville Boys' High School",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
"Life Orientation",
"Physical Sciences",
"Life Sciences",
"Information Technology",
"Computer Applications Technology",
"Accounting",
"Business Studies",
"Economics"
],
"streams": [
"Science",
"Commerce"
]
},
{
"id": "sch-005",
"province": "Western Cape",
"municipality": "Cape Town",
"name": "Rondebosch Boys' High School",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
Ln 6, Col 29 Spaces: 2 UTF-8 LF { } JSON Claude Code
Search
73%
9:35 PM
11/2/2026

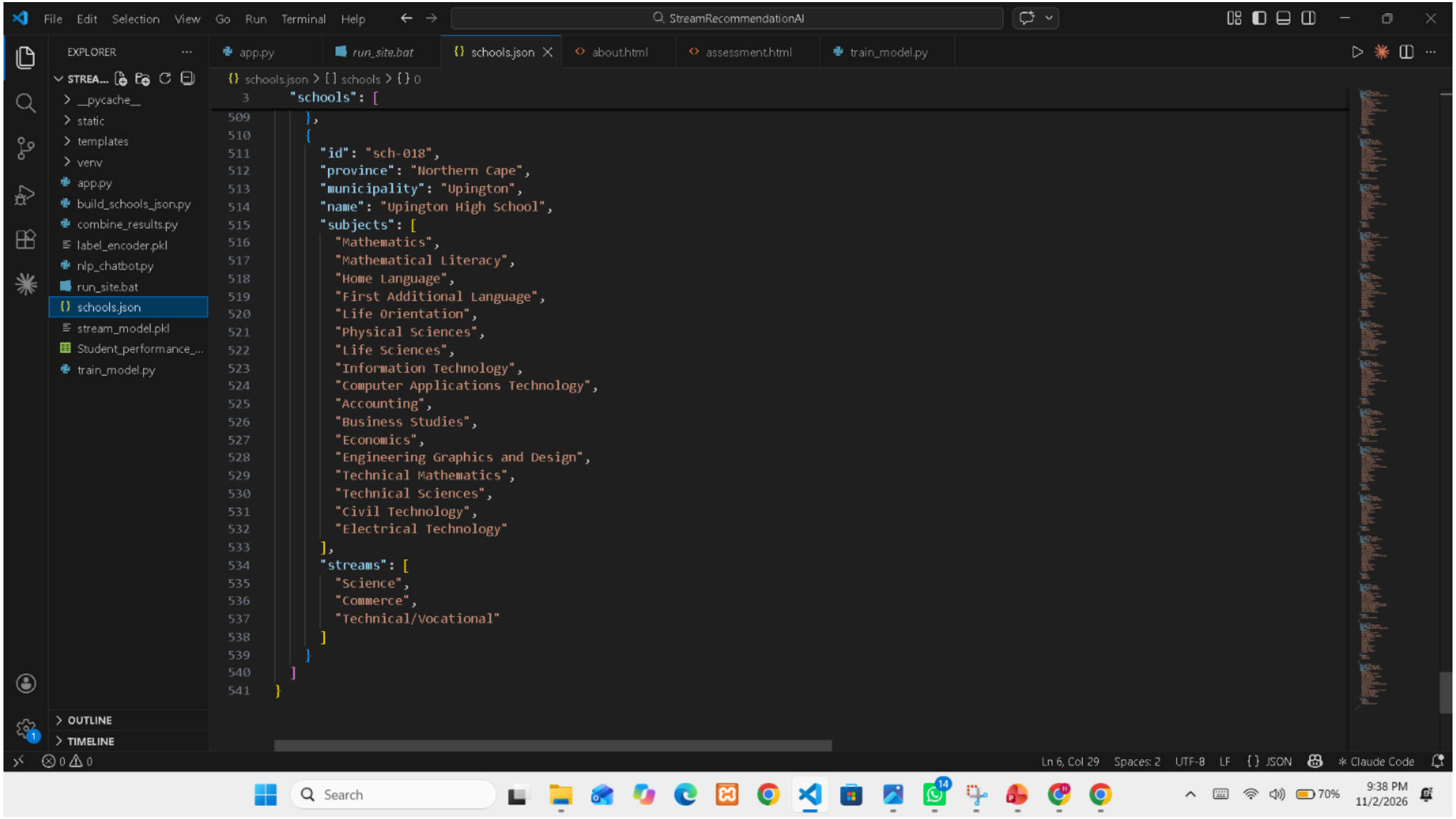

File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
schools.json > [] schools > {} 0
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
"schools": [
"subjects": [
"streams": [
"Science",
"Commerce",
"Humanities"
"id": "sch-006",
"province": "Western Cape",
"municipality": "Bellville",
"name": "Bellville High School",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
"Life Orientation",
"Physical Sciences",
"Life Sciences",
"Information Technology",
"Computer Applications Technology",
"Accounting",
"Business Studies",
"Economics",
"Engineering Graphics and Design",
"Technical Mathematics",
"Technical Sciences",
"Civil Technology",
"Electrical Technology"
"streams": [
"Science",
"Commerce",
Ln 6, Col 29
Spaces: 2
UTF-8
LF
{} JSON
Claude Code
Search
72%
9:36 PM
11/2/2026


File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
schools.json > [] schools > {} 0
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
"schools": [
"id": "sch-007",
"province": "Eastern Cape",
"municipality": "Gqeberha",
"name": "Grey High School",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
"Life Orientation",
"Physical Sciences",
"Life Sciences",
"Information Technology",
"Computer Applications Technology",
"Accounting",
"Business Studies",
"Economics",
"History",
"Geography",
"Religion Studies",
"Visual Arts",
"Dramatic Arts",
"Consumer Studies",
"Tourism"
"streams": [
"Science",
"Commerce",
"Humanities"
Ln 6, Col 29 Spaces: 2 UTF-8 LF JSON Claude Code
Search
72%
9:36 PM
11/2/2026


File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
schools.json > [] schools > {} 0
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
"schools": [
{
},
{
"id": "sch-008",
"province": "Eastern Cape",
"municipality": "Komani",
"name": "Queenstown Girls High School",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
"Life Orientation",
"Accounting",
"Business Studies",
"Economics",
"History",
"Geography",
"Religion Studies",
"Visual Arts",
"Dramatic Arts",
"Consumer Studies",
"Tourism"
],
"streams": [
"Commerce",
"Humanities"
]
},
{
"id": "sch-009",
"province": "Limpopo",
"municipality": "Mokopane",
"name": "Mokopane High School",
"subjects": [
"Mathematics",
Ln 6, Col 29
Spaces: 2
UTF-8
LF
{} JSON
Claude Code
Search
72%
9:37 PM
11/2/2026


File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
schools.json > [ ] schools > { } 0
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
"schools": [
"id": "sch-010",
"province": "Limpopo",
"municipality": "Polokwane",
"name": "Capricorn High School",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
"Life Orientation",
"Physical Sciences",
"Life Sciences",
"Information Technology",
"Computer Applications Technology",
"Engineering Graphics and Design",
"Technical Mathematics",
"Technical Sciences",
"Civil Technology",
"Electrical Technology"
"streams": [
"Science",
"Technical/Vocational"
"id": "sch-011",
"province": "Mpumalanga",
"municipality": "Mbombela",
"name": "Nelspruit High School",
"subjects": [
"Mathematics",
"Mathematical Literacy",
Ln 6, Col 29 Spaces: 2 UTF-8 LF { } JSON Claude Code
Search
71%
9:37 PM
11/2/2026


File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
schools.json > [] schools > {} 0
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
Ln 6, Col 29 Spaces: 2 UTF-8 LF { } JSON Claude Code
Search
71%
9:38 PM
11/2/2026


File Edit Selection View Go Run Terminal Help
StreamRecommendationAI
EXPLORER
app.py
run_site.bat
schools.json
about.html
assessment.html
train_model.py
__pycache__
static
templates
venv
app.py
build_schools_json.py
combine_results.py
label_encoder.pkl
nlp_chatbot.py
run_site.bat
schools.json
stream_model.pkl
Student_performance_...
train_model.py
OUTLINE
TIMELINE
{} schools.json > [] schools > {} 0
"schools": [
},
{
"id": "sch-018",
"province": "Northern Cape",
"municipality": "Upington",
"name": "Upington High School",
"subjects": [
"Mathematics",
"Mathematical Literacy",
"Home Language",
"First Additional Language",
"Life Orientation",
"Physical Sciences",
"Life Sciences",
"Information Technology",
"Computer Applications Technology",
"Accounting",
"Business Studies",
"Economics",
"Engineering Graphics and Design",
"Technical Mathematics",
"Technical Sciences",
"Civil Technology",
"Electrical Technology"
],
"streams": [
"Science",
"Commerce",
"Technical/Vocational"
]
}
]
}
Ln 6, Col 29 Spaces: 2 UTF-8 LF {} JSON Claude Code
Search
70%
9:38 PM
11/2/2026

Conclusion

The AI-Based Learner Stream Recommendation System aims to help Grade 9 learners make more informed choices about their possible Grade 10 study streams.

The prototype brings together subject marks, learner interests, machine-learning models and a web interface to provide a suggested stream.

The current results show that the system is a useful starting point, but the marks-only models need improvement. The high NLP and combined scores should be checked carefully because chatbot wording may reveal the expected answer.

In future, the system can be improved by testing with more learners, preventing data leakage, evaluating the models on separate unseen data, and gathering feedback from learners and teachers.

Overall, the project demonstrates how AI can support educational guidance, while final decisions should also consider each learner's interests and advice from teachers or career counsellors.

Thank you

Questions?